

**SVEUČILIŠTE U SPLITU  
FAKULTET ELEKTROTEHNIKE, STROJARSTVA I  
BRODOGRADNJE**

**DIPLOMSKI RAD**

**PRIMJENA STROJNOG VIDA I  
KONVOLUCIJSKIH NEURONSKIH MREŽA  
ZA PREPOZNAVANJE ČOVJEKOVIH GESTA  
S CILJEM GESTUALNOG UPRAVLJANJA  
POSTAVKAMA RAČUNALA**

**Borna Jukić**

Split, rujan 2025.



Sveučilišni diplomski studij: **Računarstvo**

Smjer/Usmjerenje: /

Oznaka programa: 250

Akadska godina: 2024./2025.

Ime i prezime: **Borna Jukić**

JMBAG: 0023138914

## ZADATAK DIPLOMSKOG RADA

Naslov: **PRIMJENA STROJNOG VIDA I KONVOLUCIJSKIH NEURONSKIH MREŽA ZA PREPOZNAVANJE ČOVJEKOVIH GESTA S CILJEM GESTUALNOG UPRAVLJANJA POSTAVKAMA RAČUNALA**

Zadatak: Istražiti i opisati inovativne pristupe upravljanja postavkama računala korištenjem kamere i tehnika strojnog učenja. Opisati dinamičko prilagođavanje postavki računala, poput miša, zvuka i sličnih parametara, putem analize korisnikovih gesta, s ciljem kreiranja intuitivnog sučelja koje unapređuje korisničko iskustvo. Prikazati teorijske osnove računalnog vida, analize slike, obrade podataka s kamere te osnovne principe strojnog učenja. Naglasak staviti na duboko učenje, odnosno na konvolucijske neuronske mreže (CNN) za prepoznavanje gesta. Osmisliti i izraditi prototip sustava koji pomoću web-kamere prikuplja podatke o gestama korisnika, te primjenom CNN-a, precizno prepoznava različite geste.

Rad predan:

Predsjednik  
Odbora za diplomski rad:

Mentor:

prof. dr. sc. Linda Vicković

prof. dr. sc. Tamara Grujić

## **IZJAVA**

Ovom izjavom potvrđujem da sam diplomski rad s naslovom „Primjena strojnog vida i konvolucijskih neuronskih mreža za prepoznavanje čovjekovih gesta s ciljem gestualnog upravljanja postavkama računala“ pod mentorstvom prof. dr. sc. Tamare Grujić pisao samostalno, primijenivši znanja i vještine stečene tijekom studiranja na Fakultetu elektrotehnike, strojarstva i brodogradnje, kao i metodologiju znanstveno-istraživačkog rada, te uz korištenje literature koja je navedena u radu. Spoznaje, stavove, zaključke, teorije i zakonitosti drugih autora koje sam izravno ili parafrizirajući naveo u diplomskom radu citirao sam i povezao s korištenim bibliografskim jedinicama.

Student

Borna Jukić

# Sadržaj

|        |                                                                         |    |
|--------|-------------------------------------------------------------------------|----|
| 1.     | Uvod .....                                                              | 1  |
| 2.     | Analiza i klasifikacija gesta ruke .....                                | 3  |
| 2.1.   | Vrste gesta ruke .....                                                  | 3  |
| 2.2.   | Tehnologije za prepoznavanje gesta .....                                | 5  |
| 2.3.   | Reprezentacija gesta u sustavima temeljenim na vizualnim uređajima..... | 8  |
| 2.4.   | Obrada podataka o gestama ruke.....                                     | 9  |
| 3.     | Analiza slike i računalni vid .....                                     | 11 |
| 3.1.   | Motivacija .....                                                        | 11 |
| 3.2.   | Osnove digitalne slike.....                                             | 11 |
| 3.3.   | Snimanje slika kamerom.....                                             | 14 |
| 3.4.   | Obrada slike.....                                                       | 16 |
| 3.4.1. | Filtriranje i normalizacija .....                                       | 16 |
| 3.4.2. | Standardizacija ulaznih podataka.....                                   | 19 |
| 3.4.3. | Napredne metode obrade.....                                             | 21 |
| 3.5.   | Definicija i svrha računalnog vida.....                                 | 22 |
| 3.6.   | Uloga računalnog vida u prepoznavanju gesta ruke .....                  | 24 |
| 3.6.1. | Detekcija.....                                                          | 25 |
| 3.6.2. | Praćenje .....                                                          | 28 |
| 4.     | Strojno i duboko učenje .....                                           | 32 |
| 4.1.   | Motivacija .....                                                        | 32 |
| 4.2.   | Strojno učenje.....                                                     | 33 |
| 4.2.1. | Vrste strojnog učenja .....                                             | 33 |
| 4.2.2. | Priprema podataka .....                                                 | 35 |
| 4.2.3. | Treniranje i evaluacija modela.....                                     | 37 |

|        |                                                                     |    |
|--------|---------------------------------------------------------------------|----|
| 4.3.   | Duboko učenje.....                                                  | 38 |
| 4.3.1. | Koncept umjetne neuronske mreže .....                               | 39 |
| 4.3.2. | Proces treniranja umjetne neuronske mreže .....                     | 42 |
| 4.3.3. | Regularizacija i hiperparametri.....                                | 43 |
| 4.4.   | Konvolucijske neuronske mreže.....                                  | 44 |
| 4.4.1. | Slojevi i osnovne komponente CNN-a.....                             | 45 |
| 4.4.2. | Uloga CNN-a u prepoznavanju gesta ruke.....                         | 48 |
| 5.     | Prototip sustava za gestualno upravljanje postavkama računala ..... | 49 |
| 5.1.   | Struktura podataka.....                                             | 51 |
| 5.2.   | CNN model .....                                                     | 53 |
| 5.3.   | Arhitektura sustava i implementacija .....                          | 57 |
| 5.4.   | Dinamičko upravljanje postavkama .....                              | 63 |
| 6.     | Evaluacija sustava za kontrolu postavki gestama ruke.....           | 67 |
| 6.1.   | Mjere točnosti.....                                                 | 67 |
| 6.2.   | Evaluacija istreniranog CNN-a .....                                 | 68 |
| 6.3.   | Prednosti i ograničenja sustava .....                               | 70 |
| 7.     | Zaključak .....                                                     | 71 |
|        | Literatura .....                                                    | 73 |
|        | Sažetak.....                                                        | 76 |
|        | Summary.....                                                        | 77 |

## 1. Uvod

U današnje vrijeme, obilježeno ubrzanim razvojem informacijskih tehnologija, sve se veća pažnja posvećuje razvoju naprednih sučelja koja omogućuju prirodniju i intuitivniju interakciju između čovjeka i računala. Tradicionalni načini upravljanja računalima, kao što su tipkovnica i miš, često ne zadovoljavaju zahtjeve suvremenih korisnika koji teže fleksibilnijim, bržim i pristupačnijim metodama komunikacije s digitalnim uređajima. U tom kontekstu, prepoznavanje gesta ruke ističe se kao jedno od najperspektivnijih rješenja, budući da omogućuje interakciju temeljenu na prirodnom pokretu i neverbalnim signalima, bez potrebe za fizičkim kontaktom ili dodatnom opremom.

Prepoznavanje gesta ruke razvilo se kao važno područje unutar ljudsko-računalne interakcije (engl. *Human-computer interaction*, skraćeno HCI). Razvoj ovakvih sustava zahtijeva integraciju različitih algoritama i tehnologija za otkrivanje, praćenje i klasifikaciju gesta. Zahvaljujući svojoj fleksibilnosti i mogućnosti prilagodbe različitim korisničkim potrebama, ovakva rješenja nalaze široku primjenu u pametnim kućnim uređajima, sustavima proširene i virtualne stvarnosti, asistivnim tehnologijama, medicinskoj dijagnostici, robotici i industrijskoj automatizaciji.

Unatoč velikom potencijalu, razvoj sustava za prepoznavanje gesta ruke suočava se s brojnim praktičnim izazovima. Jedan od ključnih izazova je varijabilnost uvjeta osvjetljenja i pozadina u kojima se sustav koristi. Promjene u intenzitetu i izvorima svjetlosti, kao i prisutnost složenih ili dinamičnih pozadina, mogu značajno otežati pouzdanu detekciju i segmentaciju ruke. Osim toga, razlike među korisnicima, poput veličine, oblika i boje kože ruke, dodatno povećavaju složenost zadatka, kao i različite poze i orijentacije ruke koje korisnici mogu spontano zauzeti tijekom interakcije.

Dodatni izazov predstavlja zahtjev za niskom latencijom, budući da korisnici očekuju trenutačan odaziv sustava na njihove pokrete, što iziskuje optimizaciju algoritama za obradu slike i klasifikaciju gesta kako bi se omogućila obrada u stvarnom vremenu. Također, potrebno je osigurati da sustav bude robustan na sitne drhtaje ruke i povremene pogrešne detekcije, čime se sprečavaju lažni pozitivni i negativni rezultati koji bi mogli otežati ili onemogućiti korisničko iskustvo. Uspješno rješavanje ovih izazova ključno je za praktičnu primjenu sustava u stvarnim uvjetima i široku prihvaćenost među korisnicima.

Motivacija za izradu ovog rada proizlazi iz potrebe za razvojem rješenja koje omogućuje učinkovito, intuitivno i dostupno upravljanje računalom pomoću gesta ruke, kao i iz želje za istraživanjem i primjenom suvremenih metoda dubokog učenja u području računalnog vida. Razvojem funkcionalnog prototipa koji prepoznaje i klasificira geste ruke u stvarnom vremenu, omogućuje se istraživanje praktičnih aspekata implementacije takvih sustava, analiza njihove učinkovitosti i identifikacija mogućih izazova i ograničenja.

Rad se sastoji od teorijskog i praktičnog dijela. U teorijskom dijelu najprije se daje pregled osnovnih pojmova i klasifikacija gesta ruke, s posebnim naglaskom na vrste gesta, načine njihove obrade te algoritme koji se koriste za njihovo prepoznavanje i klasifikaciju. Zatim se obrađuju osnove računalnog vida i obrade slike, gdje se opisuju postupci prikupljanja podataka putem kamere, metode analize i procesiranja slike te tehnike izdvajanja relevantnih značajki za kasniju klasifikaciju gesta. Nadalje, daje se pregled temeljnih principa strojnog i dubokog učenja, s posebnim osvrtom na arhitekture konvolucijskih neuronskih mreža (engl. *Convolutional Neural Networks*, skraćeno CNN) koje su se pokazale iznimno učinkovitima u zadacima prepoznavanja vizualnih uzoraka i gesta.

U praktičnom dijelu rada razvija se prototip sustava za prepoznavanje i klasifikaciju gesta ruke na temelju slike dobivene iz kamere u stvarnom vremenu. Detaljno se opisuje arhitektura sustava, uključujući proces sakupljanja i pripreme podataka, izgradnju i treniranje modela dubokog učenja, integraciju prepoznatog modela s aplikacijom za upravljanje računalom te procjena učinkovitosti razvijenog sustava na temelju niza eksperimenata. Posebna pažnja posvećuje se evaluaciji točnosti, brzine rada i robusnosti sustava pri prepoznavanju različitih gesta, kao i analizi prednosti, nedostataka i mogućih smjerova za budući razvoj.

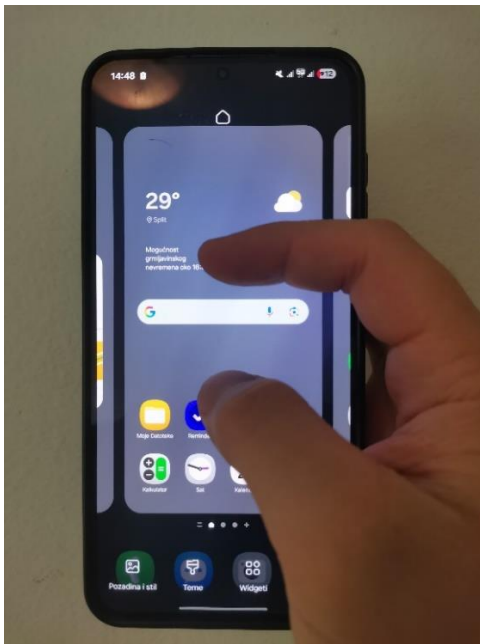
Cilj ovoga rada jest pokazati kako je primjenom suvremenih metoda računalnog vida i dubokog učenja, uz adekvatnu obradu i pripremu podataka, te pažljivo osmišljenu arhitekturu sustava, moguće izgraditi učinkovit i funkcionalan sustav za prepoznavanje i klasifikaciju gesta ruke. Očekuje se kako će rezultati ovog istraživanja doprinijeti boljem razumijevanju izazova i potencijala u razvoju naprednih sučelja za ljudsko-računalnu interakciju, kao i otvoriti mogućnosti za daljnja istraživanja i primjene u praktičnim sustavima.

## 2. Analiza i klasifikacija gesta ruke

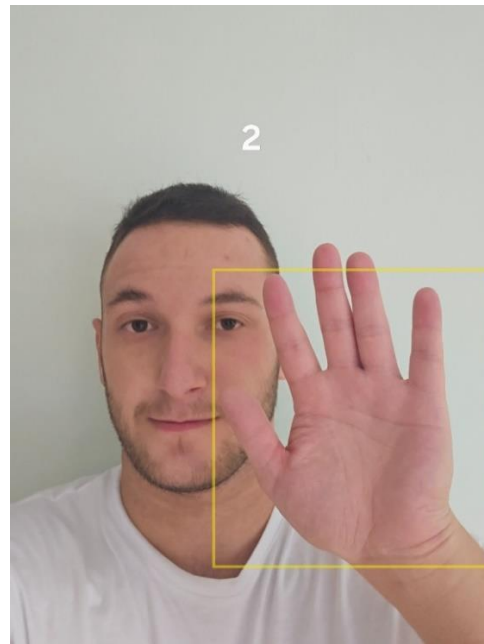
### 2.1. Vrste gesta ruke

U ovom poglavlju pojam „geste“ odnosi se isključivo na geste ruke, koje predstavljaju jedan od ključnih oblika neverbalne komunikacije, posredujući prijenos informacija, osjećaja i namjera među ljudima bez uporabe verbalnog jezika. Njihova uloga u svakodnevnoj interakciji naglašava univerzalnost gestualnog izražavanja, jer mnoge geste nadilaze jezične i kulturne barijere. Primjerice, mahanje rukom kao znak pozdrava ili pokazivanje palca u znak odobravanja prepoznatljivi su globalno. Istovremeno, određene geste imaju specifična značenja unutar pojedinih kultura ili društvenih skupina.

Razvoj digitalnih tehnologija dodatno je proširio domenu gesta, integrirajući ih kao intuitivno sredstvo interakcije između čovjeka i stroja. U suvremenoj uporabi, geste omogućuju prirodnije upravljanje računalima, pametnim uređajima poput pametnog telefona na slici 2.1. i aplikacijama poput aplikacije kamere na slici 2.2. Na taj se način smanjuje potreba za fizičkim kontaktom i unaprjeđuje korisničko iskustvo.



Slika 2.1. Kontrola pametnog telefona gestom



Slika 2.2. Kontrola aplikacije kamere gestom

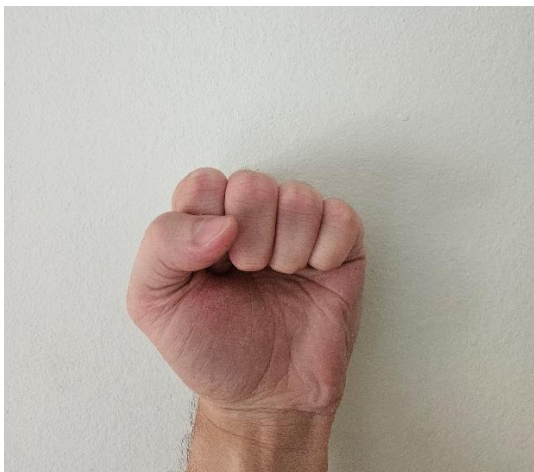


U kontekstu računalne obrade i prepoznavanja, geste se sistematiziraju i analiziraju s ciljem izgradnje pouzdanih sustava koji mogu interpretirati ljudske pokrete. Prva i najvažnija podjela gesti temelji se na njihovoj vremenskoj strukturi, odnosno razlikovanju statičkih i dinamičkih gesta.

Statičke geste definiraju se kao stabilni i nepromjenjivi položaji ruke ili prstiju, koji se zadržavaju u određenom trenutku vremena bez vidljivih promjena, kao što se vidi na slici 2.3. Kod ovakvih gesta poseban naglasak stavlja se na preciznu definiciju oblika i konfiguracije, što omogućuje pouzdanu identifikaciju pojedinih naredbi ili simbola. Primjena takvih gestualnih izraza osobito je česta u komunikaciji znakovnim jezikom te u upravljanju multimedijским uređajima. Njihova glavna prednost leži u jednostavnosti detekcije jer obrada zahtijeva analizu samo jednog kadra slike, bez potrebe za praćenjem vremenske sekvence.

S druge strane, dinamičke geste uključuju pokrete koji se odvijaju tijekom određenog vremenskog intervala, što znači da su definirane ne samo konačnim položajem nego i putanjom, brzinom i redoslijedom pokreta. Takve geste obuhvaćaju, primjerice, mahanje rukom za pozdrav, pokazivanje smjera ili izvođenje složenijih sekvenci koje se koriste u znakovnom jeziku, kao što je prikazano na slici 2.4.

Za razliku od statičkih, dinamičke geste zahtijevaju analizu više uzastopnih kadrova, odnosno praćenje promjena kroz vrijeme, što čini njihovo prepoznavanje računalno zahtjevnijim i osjetljivijim na varijacije u izvođenju pokreta [1]. Slijedom toga, takve geste omogućuju izražavanje kompleksnijih naredbi i informacija pa su posebno važne u suvremenim interaktivnim sustavima, proširenoj stvarnosti i robotici, gdje omogućuju prirodnu i bogatiju interakciju između korisnika i uređaja.



*Slika 2.3. Statička gesta*



*Slika 2.4. Dinamička gesta*

Ova temeljna podjela ima značajan utjecaj na odabir metoda za detekciju, obradu i klasifikaciju gesta te definira izazove i mogućnosti u razvoju sustava za automatsko prepoznavanje gesta.

## **2.2. Tehnologije za prepoznavanje gesta**

Razvoj preciznih i učinkovitih sustava za prepoznavanje gesta ruke izravno je povezan s kontinuiranim napretkom modernih tehnologija. Ključnu ulogu u tome imaju senzorni sustavi koji omogućuju automatizirano bilježenje i interpretaciju ljudskih pokreta. Ovisno o vrsti senzora koji se koristi za detekciju gesta, tehnologije za prepoznavanje mogu se klasificirati u dvije glavne skupine: kontaktne uređaje (engl. *contact-based devices*) i vizualne uređaje (engl. *vision-based devices*). U nastavku su detaljno opisane obje kategorije, uključujući njihove podjele i međusobne razlike.

Kontaktne uređaje za prepoznavanje gesta ruke predstavljaju tehnologije koje koriste izravni fizički kontakt između korisnika i senzorskih površina za bilježenje i interpretaciju pokreta. Ovi uređaji detektiraju gestu ruke tako što prate promjene u položaju, savijanju prstiju ili pritisku na površinu, najčešće pomoću integriranih senzora poput senzora savijanja, sile ili inercijskih mjernih jedinica (engl. *Inertial Measurement Units*, skraćeno IMU). Najčešća primjena ovih uređaja je u

područjima gdje je potrebna visoka preciznost, poput virtualne stvarnosti, medicinske rehabilitacije i napredne robotike.

Kontaktne sustavi najčešće se realiziraju kroz senzorske rukavice (engl. *data gloves*), koje se prema vrsti ugrađenih senzora dijele na nekoliko kategorija [2]:

- Mehaničke rukavice (npr. CyberGlove III na slici 2.5.) - koriste fizičke senzore, poput potencijometara, ugrađene u spojeve rukavice kako bi precizno mjerile kutove savijanja prstiju i položaj šake.
- Inercijske rukavice (npr. MoCap Glove na slici 2.6.) - koriste akcelerometre, žiroskope i magnetometre za detekciju ubrzanja, rotacije i orijentacije ruke u prostoru.
- Elektromagnetske rukavice - koriste magnetske senzore za određivanje položaja i kretanja ruke unutar elektromagnetskog polja.

Glavne prednosti kontaktnih uređaja su visoka preciznost i otpornost na vanjske smetnje, dok su njihovi nedostaci viša cijena, potreba za redovitom kalibracijom i moguća nelagoda pri dugotrajnom nošenju.



Slika 2.5. Mehanička rukavica CyberGlove III  
[3]



Slika 2.6. Inercijska rukavica MoCap Glove  
[3]

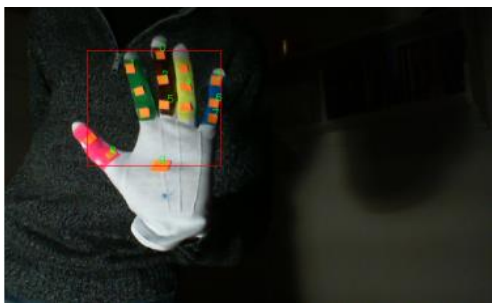
Vizualni uređaji za prepoznavanje gesta ruke omogućuju detekciju i interpretaciju pokreta bez fizičkog kontakta između korisnika i senzorskog sustava. Ova tehnologija koristi kamere i napredne algoritme računalnog vida za praćenje oblika, položaja i kretanja ruke u stvarnom

vremenu, omogućujući tako prirodniju i intuitivniju interakciju s računalima i drugim uređajima. Posebno su korisni u situacijama gdje je poželjno da korisnik ima potpunu slobodu pokreta i ne mora nositi dodatnu opremu.

Sustavi temeljeni na vizualnim uređajima najčešće se dijele prema vrsti kamere i načinu obrade slike na nekoliko kategorija:

- Sustavi s običnim tj. RGB kamerama - analiziraju dvodimenzionalne slike u boji te koriste metode za detekciju oblika, boje kože ili kontura ruke.
- Sustavi s dubinskim kamerama (npr. Microsoft Kinect na slici 2.8.) - omogućuju trodimenzionalno praćenje ruke koristeći podatke o dubini scene, čime se znatno povećava preciznost detekcije i smanjuje utjecaj pozadine.
- Metode s markerima [4] - koriste vizualno prepoznatljive oznake postavljene na ruku, kao na slici 2.7., kako bi se olakšalo praćenje pokreta.

Glavne prednosti ovakvih uređaja uključuju potpunu slobodu kretanja za korisnika, mogućnost višekorisničke interakcije te jednostavnu integraciju u svakodnevne uređaje poput pametnih telefona, računala i interaktivnih ekrana. S druge strane, izazovi uključuju osjetljivost na uvjete osvjetljenja, smetnje u pozadini i potrebu za značajnom računalnom obradom, što može ograničiti njihovu primjenu u stvarnom vremenu.



*Slika 2.7. Praćenje pokreta metodom markera*  
[5]



*Slika 2.8. Microsoft Kinect dubinska kamera*  
[6]

### 2.3. Reprezentacija gesta u sustavima temeljenim na vizualnim uređajima

Reprezentacija gesta u sustavima temeljenim na vizualnim uređajima najčešće se ostvaruje primjenom dvodimenzionalnih (2D) ili trodimenzionalnih (3D) modela. Dvodimenzionalni modeli još se nazivaju i modeli temeljeni na izgledu (engl. *Appearance-based models*) [7].

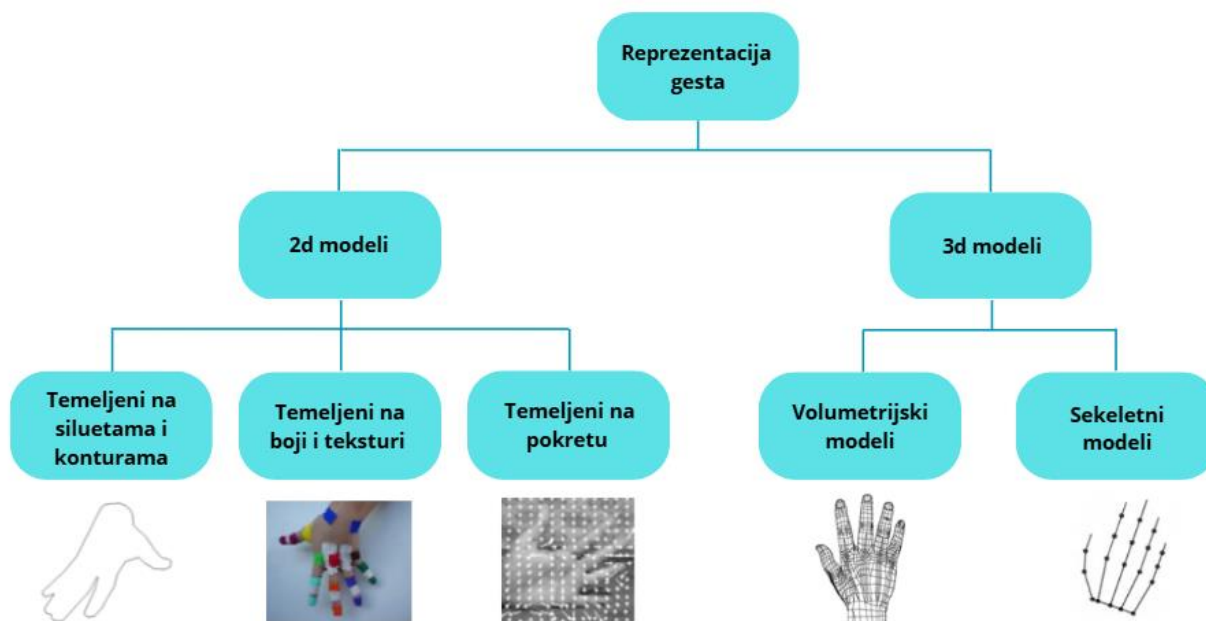
Pristup temeljen na izgledu koristi informacije iz dvodimenzionalnih slika za prepoznavanje gesta ruke. Ovaj pristup se oslanja na vizualne značajke, poput oblika, kontura, boje kože i teksture, koje se izdvajaju iz slike i analiziraju kako bi se opisala određena gesta. Ove metode popularne su zbog jednostavne implementacije i brzine obrade, a najčešće koriste sljedeće podvrste modela:

- Modeli temeljeni na konturama i siluetama - fokusiraju se na prepoznavanje obrisa ruke i njezina položaja kroz analizu kontura, rubova ili silueta na slici.
- Modeli temeljeni na boji i teksturi - koriste informacije o boji kože ili teksturi površine ruke za segmentaciju i prepoznavanje, što može biti korisno za razlikovanje ruke od pozadine.
- Modeli temeljeni na pokretu - analiziraju promjene u položaju ili izgledu ruke kroz vremenski slijed dvodimenzionalnih slika.

Trodimenzionalni (3D) modeli predstavljaju ruku kao strukturu u prostoru, koristeći podatke o dubini i prostornom položaju prstiju i dlana. Ovi modeli omogućuju realističniji prikaz i analizu složenijih gesti, neovisno o kutu snimanja ili orijentaciji ruke. U 3D pristupu postoje dvije glavne vrste modela:

- Volumetrijski modeli - opisuju volumen ili površinu ruke, često koristeći 3D skeniranje ili višekutne kamere.
- Skeletni modeli - prikazuju osnovne zglobove i kosti ruke kao niz povezanih točaka, što omogućuje precizno praćenje pokreta prstiju i dlana u prostoru.

Izbor između 2D i 3D modela ovisi o potrebama primjene, dostupnoj tehnologiji, složenosti gesta, uvjetima okoline te zahtjevima za brzinom i preciznošću prepoznavanja. Detaljna podjela ovih pristupa prikazana je na slici 2.9.



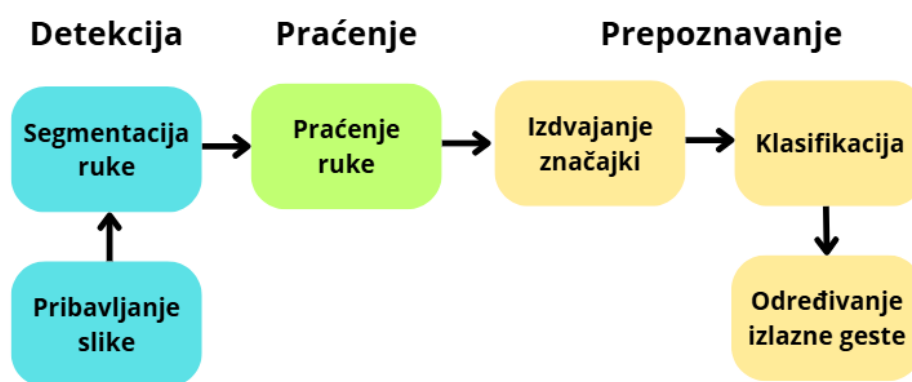
*Slika 2.9. Reprezentacija gesta u sustavima temeljenim na vizualnim senzorima [7]*

U nastavku rada, pod obradom podataka i prepoznavanjem gesta ruke podrazumijevat će se korištenje sustava temeljenih na vizualnim uređajima, odnosno računalnom vidu.

## 2.4. Obrada podataka o gestama ruke

Učinkovita analiza i automatsko prepoznavanje gesta ruke temelje se na jasno definiranim fazama obrade podataka. Svaki sustav za prepoznavanje gesta prolazi kroz niz koraka koji omogućuju precizno izdvajanje, praćenje i interpretaciju pokreta ruke. Obrada podataka o gestama najčešće se dijeli na tri ključne faze: detekciju, praćenje i prepoznavanje. Svaka od ovih faza ima specifične izazove i zahtijeva primjenu odgovarajućih algoritama i tehnika kako bi se osigurala visoka preciznost i robusnost sustava u stvarnim uvjetima rada.

Unutar faze detekcije, najvažniji koraci su pribavljanje slike (engl. *image acquisition*) i segmentacija ruke (engl. *hand segmentation*), čime se iz ulaznog podatka izdvajaju relevantni dijelovi slike za daljnju obradu. Faza praćenja obuhvaća praćenje ruke (engl. *hand tracking*) kroz niz uzastopnih kadrova, čime se bilježi dinamika pokreta. U fazi prepoznavanja posebnu ulogu imaju izdvajanje značajki (engl. *feature extraction*), klasifikacija i određivanje izlazne geste (engl. *output gesture*), što omogućuje konačno prepoznavanje i interpretaciju pokreta. Detaljna podjela ovih faza i njihovih podkoraka prikazana je na slici 2.10. [8].



Slika 2.10. Faze sustava prepoznavanja gesta ruke [8]

S obzirom na važnost svake pojedine faze; od detekcije i praćenja ruke, preko izdvajanja značajki, do konačne klasifikacije i interpretacije geste, u nastavku rada detaljno će biti objašnjeni najčešće korišteni algoritmi i metode za svaku od faza. Time će se omogućiti dublje razumijevanje načina na koji svaka faza doprinosi uspješnom automatskom prepoznavanju gesta ruke u stvarnom vremenu i raznim okruženjima.

### **3. Analiza slike i računalni vid**

#### **3.1. Motivacija**

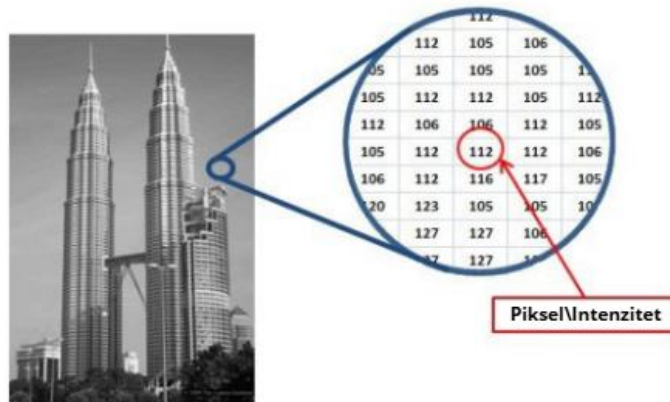
Prije nego što sustav prepoznavanja gesta može započeti s detekcijom i analizom pokreta ruke, nužno je osigurati kvalitetnu i pouzdanu ulaznu informaciju u obliku digitalne slike. Svaka faza automatskog prepoznavanja gesta započinje upravo prikupljanjem slike putem kamere, pri čemu je važno osigurati odgovarajuću rezoluciju, osvjetljenje i fokus. Nakon prikupljanja slike, slijedi njezina obrada kako bi se iz sirovih podataka izdvojile ključne vizualne informacije i uklonili neželjeni elementi, poput šuma i smetnji iz pozadine. Ovi inicijalni koraci predstavljaju temelj za sve daljnje faze računalne obrade i prepoznavanja, jer kvaliteta i relevantnost podataka izravno utječu na uspješnost detekcije i točnost interpretacije gesta ruke. Stoga je važno detaljno razumjeti kako se digitalna slika prikuplja, koje su osnovne karakteristike digitalne slike i koje se metode najčešće primjenjuju za njezinu osnovnu obradu prije ulaska u složenije faze sustava.

#### **3.2. Osnove digitalne slike**

Digitalna slika predstavlja numeričku, diskretnu aproksimaciju stvarne scene i sastoji se od osnovnih jedinica nazvanih piksela, koji su raspoređeni u pravokutnu mrežu, odnosno matricu, kao što se vidi na slici 3.1. Svaki piksel sadrži numeričku vrijednost koja opisuje svjetlinu ili boju na određenoj poziciji. Temeljne karakteristike digitalne slike su njezina razlučivost i dubina zapisa.

Razlučivost označava broj piksela po širini i visini slike te određuje količinu detalja koju slika može prikazati gdje veći broj piksela omogućuje prikaz finijih vizualnih informacija. Dubina zapisa predstavlja broj bitova dodijeljenih svakom pikselu, čime se određuje koliko različitih vrijednosti svjetline ili boje svaki piksel može prikazati. Kod jednostavnih, crno-bijelih slika, svaki piksel opisan je jednom numeričkom vrijednošću, dok su kod slika u boji pikseli definirani s više vrijednosti, po jednoj za svaki kanal boje ili svjetlosti. Kombinacija razlučivosti i dubine zapisa određuje ukupnu kvalitetu slike, njezinu mogućnost prikaza detalja i bogatstvo prikazanih nijansi ili boja [9].



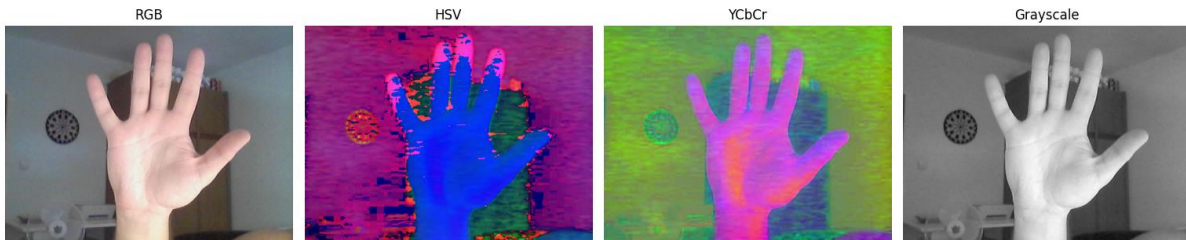


Slika 3.1. Prikaz digitalne slike kao matrice piksela [10]

Za prikaz boja u digitalnim slikama koriste se različiti modeli boja kao što se vidi na slici 3.2., od kojih su najčešći:

- RGB model - temelji se na aditivnom miješanju triju osnovnih boja: crvene (engl. *Red*), zelene (engl. *Green*) i plave (engl. *Blue*). Svaka boja na slici definira se kao kombinacija intenziteta tih triju komponenti, najčešće u rasponu od 0 do 255 za 8 bitova po kanalu. Sve moguće kombinacije stvaraju puni spektar boja koji se može prikazati na digitalnim uređajima. RGB model je standard za digitalne fotoaparate, monitore i razne multimedijske aplikacije.
- HSV model - organizira boje prema ljudskom poimanju: nijansa (eng. *Hue*) određuje osnovnu boju (npr. crvena, plava, žuta), zasićenost (engl. *Saturation*) određuje intenzitet ili čistoću boje, a vrijednost (engl. *Value*) označava svjetlinu. Ovaj model je koristan za aplikacije koje zahtijevaju manipulaciju i odabir boje jer omogućuje intuitivnije prilagodbe nego RGB.
- YCbCr model - odvaja svjetlinu, označenu kao Y, od informacija o boji koje su predstavljene dvama komponentama: Cb, koja opisuje razliku između plave i žute, te Cr, koja opisuje razliku između crvene i plave boje. Ovaj model je široko korišten u video-kompresiji (MPEG, JPEG) i televizijskoj industriji jer omogućuje učinkovitu manipulaciju i kompresiju slike smanjenjem količine boje, a uz očuvanje detalja svjetline.

- Siva ljestvica (engl. *greyscale*) - predstavlja sliku pomoću jednog kanala gdje svaki piksel ima vrijednost koja označava intenzitet svjetline između crne i bijele boje. Najčešće koristi 8 bitova po pikselu, što omogućuje prikaz 256 razina sive boje. Ovaj model je jednostavniji za obradu i često se koristi kao prvi korak u algoritmima računalnog vida.



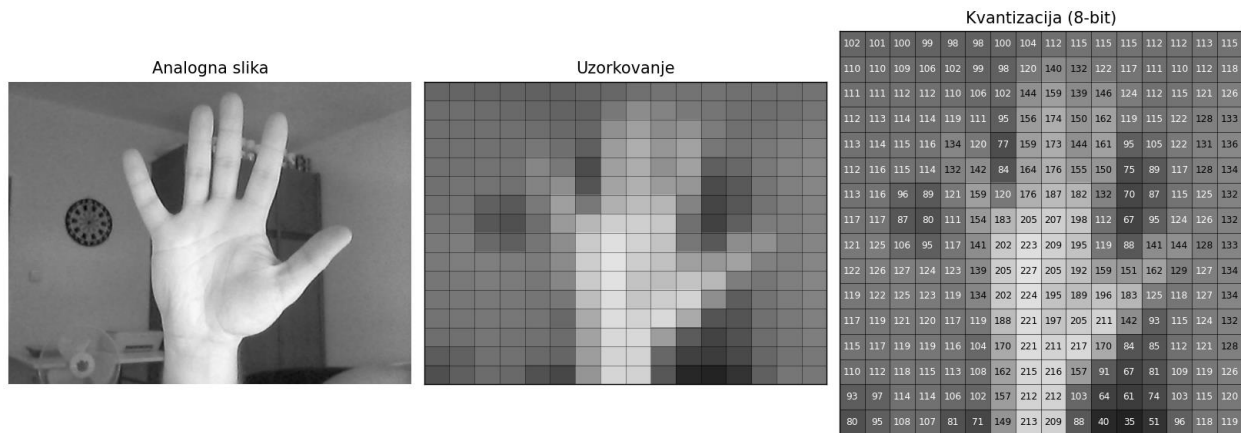
*Slika 3.2. Prikaz modela boja*

Prije nego što stvarnu, kontinuiranu scenu možemo obraditi računalom, potrebno ju je pretvoriti u digitalni oblik. Taj proces uključuje dva ključna koraka: uzorkovanje i kvantizaciju. Ovi postupci omogućuju da neprekidni vizualni podaci, kakve opažamo u stvarnosti, postanu skup jasno određenih i numerički opisanih vrijednosti u digitalnoj slici.

Uzorkovanje je proces pretvaranja kontinuiranog vizualnog prostora ili scene u konačan, diskretan skup točaka, odnosno piksela. Svaki piksel predstavlja uzorak sa specifične pozicije na slici. Gustoća uzorkovanja određuje razlučivost slike; veći broj uzoraka rezultira slikom s više detalja, ali i većom količinom podataka.

Kvantizacija je proces pretvaranja kontinuiranih vrijednosti svjetlosti ili boje svakog piksela u diskretne razine. Na primjer, kod 8-bitne kvantizacije, svjetlina ili svaka komponenta boje može imati 256 mogućih vrijednosti. Fina kvantizacija (veći broj bitova) omogućuje glađe prijelaze između nijansi i manje vidljive prijelaze.

Na slici 3.3. prikazan je rezultat procesa uzorkovanja i kvantizacije s 8 bitova, gdje je kontinuirana scena pretvorena u diskretnu mrežu piksela. Zahvaljujući jasno vidljivim kvadratima matrice i rasponu nijansi, prepoznaje se obris ruke unutar digitalne slike.



Slika 3.3. Proces uzorkovanja i 8-bitne kvantizacije

Digitalna slika, osim idealnih podataka, najčešće sadrži i neželjene elemente koji nastaju zbog ograničenja senzora, vanjskih uvjeta i nedostataka optike. Šum se pojavljuje kao nasumične varijacije svjetline ili boje, a uzroci mogu biti toplinski šum senzora, elektronički šum ili loši uvjeti osvjetljenja. Izobličenja uključuju efekte poput zamućenja zbog pokreta ili fokusa, geometrijske deformacije uzrokovane lećama [11], kao i pogreške uzrokovane kompresijom slike. Navedeni problemi mogu otežati ili smanjiti točnost naknadne obrade i detekcije gesta, pa je njihovo prepoznavanje i smanjenje iznimno važno za pouzdane sustave prepoznavanja gesta.

Ove temeljne karakteristike digitalne slike i realni izazovi u njezinoj akviziciji čine osnovu za razumijevanje procesa prikupljanja slike s kamere, koji je sljedeći ključan korak u sustavu za prepoznavanje gesta.

### 3.3. Snimanje slika kamerom

Proces prikupljanja digitalne slike započinje snimanjem stvarne scene pomoću kamere, gdje ključnu ulogu imaju senzori slike, najčešće CCD (engl. *Charge Coupled Device*) i CMOS (engl. *Complementary Metal Oxide Semiconductor*). CCD senzori poznati su po visokoj kvaliteti slike i osjetljivosti na svjetlo te se često koriste u profesionalnim digitalnim fotoaparatom, znanstvenim i astronomskim kamerama. CMOS senzori energetski su učinkovitiji i omogućuju veću integraciju s drugim elektroničkim komponentama, zbog čega su danas češće prisutni u modernim digitalnim kamerama i pametnim telefonima. Ovi senzori pretvaraju optički signal, odnosno svjetlost koja

dolazi do senzora, u električni signal. Potom se analogni električni signal digitalizira, na način objašnjen u poglavlju iznad, čime nastaje niz vrijednosti koji računalni sustav može dalje obrađivati, najčešće u obliku RGB slike.

Kvaliteta dobivene slike uvelike ovisi o hardverskim parametrima kamere, među kojima su ključni:

- Fokus - određuje koliko su objekti u sceni oštri ili zamagljeni. Pravilno podešen fokus omogućuje jasno prepoznavanje detalja na slici, dok neprecizan fokus rezultira zamućenim prikazom.
- Ekspozicija - predstavlja količinu svjetlosti koja dopire do senzora tijekom snimanja i određuje svjetlinu slike. Nedovoljna ekspozicija uzrokuje tamne, a prevelika ekspozicija svijetle i izbljedjele slike.
- Brzina kadrova – određuje ju broj slika koje kamera može zabilježiti u jednoj sekundi, a izražava se u fps (engl. *frames per second*). Važna je pri snimanju brzih ili dinamičnih scena, jer veća brzina smanjuje zamućenje pokreta i omogućuje preciznije bilježenje promjena u vremenu [11].

Osim navedenih hardverskih parametara, i uvjeti osvjetljenja u prostoru značajno utječu na kvalitetu slike. Neadekvatno ili neravnomjerno osvjetljenje može uzrokovati pojavu šuma, isprane ili tamne dijelove slike te gubitak detalja, čime se otežava kasnija obrada i prepoznavanje informacija na snimci. Pravilno podešavanje svih ovih parametara ključno je za postizanje kvalitetne i upotrebljive digitalne slike.

S obzirom na to da sustavi za prepoznavanje gesta ruke većinom koriste CMOS senzore, kao i na utjecaj svih navedenih parametara koji mogu uzrokovati pojavu šuma, zamućenja i geometrijskih distorzija, već u fazi prikupljanja slike često se javlja potreba za primjenom metoda korekcije i poboljšanja slike. Time se omogućuje što preciznija naknadna analiza i obrada, čime se povećava pouzdanost i učinkovitost cijelog sustava za prepoznavanje gesta.

### 3.4. Obrada slike

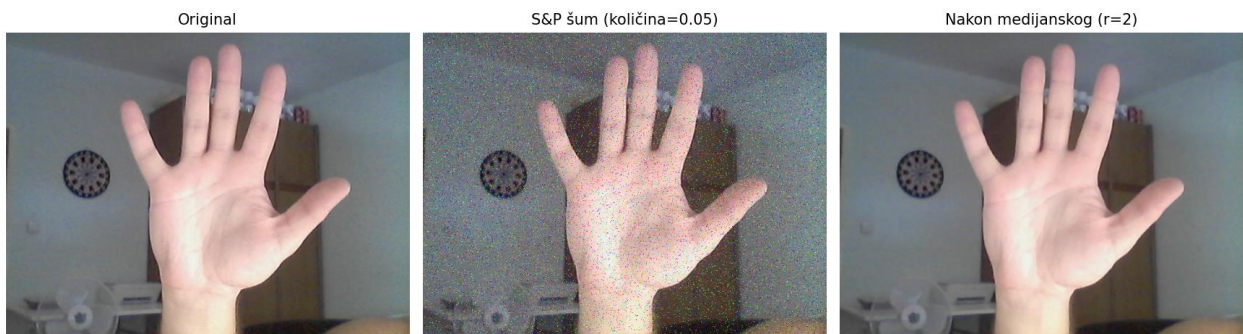
Problemi poput šuma, zamućenja, nepravilnog osvjetljenja i geometrijskih distorzija, spomenuti u prethodnom poglavlju, mogu značajno smanjiti kvalitetu digitalne slike i otežati njezinu pouzdanu analizu. Kako bi se ti nedostaci uklonili ili barem ublažili, prije daljnje obrade potrebno je provesti niz postupaka poznatih pod skupnim nazivom pretprocesiranje slike (engl. *preprocessing*). U okviru ovog rada, pretprocesiranje će biti podijeljeno na tri osnovne skupine: filtriranje i normalizaciju, standardizaciju ulaznih podataka i napredne metode obrade slike. Ovakva podjela omogućuje sustavno sagledavanje svih ključnih koraka potrebnih za pripremu slike u sustavima za prepoznavanje gesta ruke.

#### 3.4.1. Filtriranje i normalizacija

Šum u slici često se uklanja primjenom specijaliziranih filtara, od kojih su najčešće korišteni medijanski filter i Gaussov filter [12].

Medijanski filter je nelinearni filter koji je osobito učinkovit za uklanjanje impulsnog šuma, poznatog kao „sol i papar“ šum. To je vrsta digitalnog šuma koji se manifestira pojavom nasumično raspoređenih vrlo svijetlih („sol“) i vrlo tamnih („papar“) piksela na slici. Ovakav šum najčešće nastaje zbog smetnji tijekom prijenosa ili digitalizacije slike, kvarova senzora ili grešaka u komunikaciji, a karakteristično je što pojedini pikseli značajno odstupaju od vrijednosti svojih susjeda.

Princip rada filtra temelji se na određivanju prozora (npr. 3 x 3 ili 5 x 5 piksela) oko svakog piksela slike, unutar kojeg se vrijednosti piksela sortiraju po iznosu, a ona u sredini iz tog skupa (medijan) zamjenjuje izvornu vrijednost piksela. Na taj način, ekstremne vrijednosti koje odskaču od većine (vrlo svijetli ili tamni pikseli) bivaju uklonjene, dok se struktura i oštrina rubova u slici uglavnom zadržavaju. Medijanski filter se tako pokazuje vrlo učinkovitim alatom za očuvanje detalja i kontura slike kod uklanjanja impulsnog šuma što se vidi na slici 3.4.



*Slika 3.4. Primjena medijanskog filtra na sliku*

Gaussov filtar je jedan od najčešće korištenih linearnih filtara za zaglađivanje slike. Temelji se na Gaussovoj funkciji (normalnoj raspodjeli), koja određuje koliki utjecaj imaju vrijednosti piksela u okolini promatranog piksela prilikom izračuna nove vrijednosti tog piksela. Gaussova funkcija ima oblik zvona, a njezina standardna devijacija određuje širinu i intenzitet zaglađivanja. Veća vrijednost standardne devijacije rezultira jačim zaglađivanjem i širim utjecajem okolnih piksela.

Kod primjene Gaussovog filtra, za svaki piksel slike definira se prozor (tipično  $3 \times 3$ ,  $5 \times 5$ ,  $7 \times 7$  ili veći), unutar kojeg se izračunava nova vrijednost kao prosjek svih piksela iz prozora, pri čemu pikseli najbliži središtu prozora (odnosno promatranom pikselu) imaju najveći utjecaj na rezultat, dok utjecaj udaljenijih piksela brzo opada prema rubovima prozora. Na taj način, Gaussov filtar učinkovito smanjuje visokofrekventni šum, poput nasumičnih sitnih odstupanja intenziteta, i izjednačava prijelaze između različitih područja slike.

Zbog tog svojstva, Gaussov filtar se često koristi kao priprema za kasniju detekciju rubova, jer pomaže u uklanjanju šuma koji bi mogao uzrokovati lažne ili nepotrebne rubove. Međutim, budući da se vrijednosti s obje strane ruba međusobno „miješaju“ kroz postupak filtriranja, filtar može dovesti do blagog zamućenja rubova i smanjenja oštrote kontura kao što je vidljivo na slici 3.5. To znači da će granice između objekata na slici biti manje izražene, osobito ako se koristi veći prozor ili veća standardna devijacija.





*Slika 3.5. Primjena Gaussovog filtra na sliku*

Nakon uklanjanja šuma, često je potrebno dodatno poboljšati vizualnu kvalitetu slike kako bi se olakšala naknadna analiza. Najvažniji koraci u tom smislu su poboljšanje kontrasta i normalizacija osvjetljenja, kojima se ističu detalji i osigurava ujednačen prikaz svjetline bez obzira na varijacije u uvjetima snimanja.

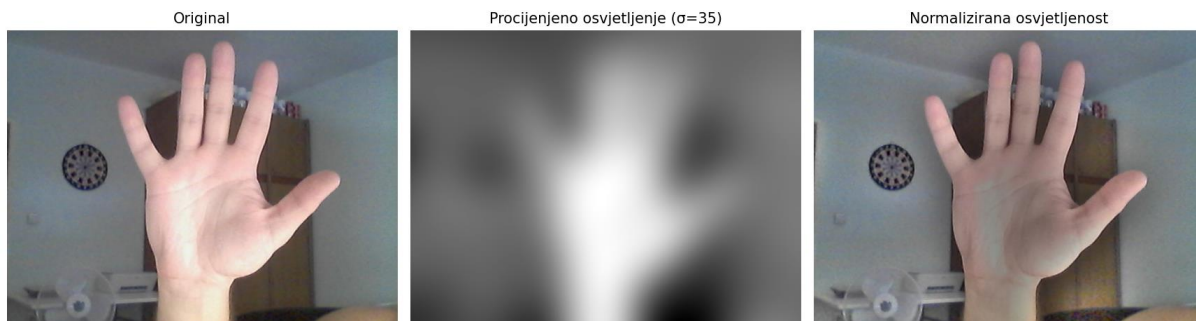
Poboljšanje kontrasta provodi se kako bi se naglasili detalji na slici, osobito u područjima koja su inače presvijetla ili pretamna. Jedna od najčešće korištenih metoda je ekvalizacija histograma (engl. *histogram equalization*). Ekvalizacija histograma redistribuira intenzitete piksela tako da što više iskoristi cijeli dinamički raspon svjetline slike. Prvo se izračuna histogram slike, koji prikazuje koliko piksela ima svaku razinu svjetline. Zatim se pomoću funkcije kumulativne raspodjele (CDF) svjetlina piksela na slici preraspodjeljuje tako da rezultat bude ravnomjernija raspodjela intenziteta po cijelom rasponu vrijednosti. Time se povećava kontrast u tamnim i svijetlim područjima, što olakšava prepoznavanje detalja kao što je vidljivo na slici 3.6.



*Slika 3.6. Proces poboljšanja kontrasta*

Normalizacija osvjetljenja ima za cilj smanjiti utjecaj neravnomjernog ili promjenjivog osvjetljenja na slici, čime se postiže ravnomjerniji prikaz svjetline u svim dijelovima slike, kao što se vidi na slici 3.7. U tu svrhu primjenjuju se adaptivne metode koje analiziraju raspodjelu svjetline na slici i automatski prilagođavaju intenzitet svakog piksela kako bi se postigla uravnoteženost svjetline. Takav pristup omogućuje kompenzaciju za sjene, refleksije i druge promjene u uvjetima osvjetljenja, pa dijelovi slike koji su prethodno bili previše tamni ili svijetli postaju bolje usklađeni s ostatkom slike.

Naprednije metode uključuju lokalnu normalizaciju, gdje se svjetlina svakog piksela prilagođava na temelju vrijednosti u njegovoj okolini. Time se kompenziraju sjene, refleksije i lokalne promjene u osvjetljenju. Ove metode često koriste statističke mjere poput srednje vrijednosti i standardne devijacije u lokalnom prozoru kako bi se intenziteti prilagodili lokalnom kontekstu.



*Slika 3.7. Proces normalizacije osvjetljenja*

#### 3.4.2. Standardizacija ulaznih podataka

Za pouzdanu i usporedivu obradu u sustavima za prepoznavanje gesta ruke, iznimno je važno da sve slike koje ulaze u sustav imaju ujednačene tehničke karakteristike. Različiti izvori slike, kamere ili uvjeti snimanja mogu rezultirati varijacijama u veličini slike, formatu, orijentaciji ili prisutnosti tehničkih nedostataka, što može otežati ili onemogućiti učinkovitu analizu [9].

Standardizacija slike obuhvaća nekoliko ključnih koraka:



- Normalizacija veličine slike - sve slike se prevode na istu rezoluciju, primjerice 256x256 piksela. Time se osigurava konzistentnost dimenzija, što je nužno za algoritme koji zahtijevaju fiksni ulazni format i olakšava usporedbu rezultata između različitih uzoraka.
- Prilagodba formata - slike se mogu pretvoriti u standardizirani format, poput RGB ili grayscale formata. Odabir formata ovisi o zahtjevima algoritma i ciljevima analize. Primjerice, neki pristupi koriste samo informacije o svjetlini, dok je za druge važno očuvati boju.
- Korekcija nedostataka - prije analize potrebno je ukloniti ili ispraviti moguće tehničke nedostatke slike, poput geometrijskih izobličenja (deformacija uzrokovanih optikom), neželjenih pomaka, praznih područja ili nepotrebnog obruba. U tu svrhu koriste se metode kao što su geometrijska registracija, izrezivanje (engl. *cropping*), popunjavanje praznih dijelova i ispravljanje perspektive.

Na slici 3.8. prikazani su najvažniji koraci za korekciju tehničkih nedostataka digitalne slike. Geometrijska registracija podrazumijeva usklađivanje dviju ili više slika tako da se svi ključni objekti ili točke poklapaju u koordinatnom sustavu. Popunjavanje praznih dijelova odnosi se na interpolaciju ili nadomještanje nedostajućih piksela u slikama s oštećenjima ili nedostacima. Ispravljanje perspektive omogućuje pretvaranje slike snimljene pod kutom u prikaz kao da je snimljena ravno, čime se uklanjaju deformacije nastale zbog kuta snimanja. Izrezivanje se koristi za uklanjanje nepotrebnih rubova ili fokusiranje na područje od interesa unutar slike.



Slika 3.8. Proces korekcije tehničkih nedostataka

Provođenjem ovih koraka standardizacije, svi ulazni podaci postaju usporedivi i algoritmi za prepoznavanje gesta mogu na njih jednako učinkovito primijeniti svoje metode, neovisno o izvoru slike ili različitim uvjetima snimanja.

### 3.4.3. Napredne metode obrade

Nakon filtriranja, normalizacije i standardizacije ulaznih podataka, ključni korak u pripremi slike za prepoznavanje gesta ruke je detekcija rubova i značajki. Ova metoda omogućuje izdvajanje najvažnijih informacija o obliku, konturama i karakterističnim točkama na slici, čime se znatno olakšava identifikacija i praćenje gesta.

Detekcija rubova ima za cilj pronalazak područja u slici gdje dolazi do naglih promjena intenziteta svjetline ili boje, odnosno gdje postoje prijelazi između različitih objekata ili površina. Najčešće korišteni algoritmi su [12]:

- Sobelov operator - izdvaja rubove tako da izračunava gradijent svjetline u horizontalnom i vertikalnom smjeru, pri čemu gradijent predstavlja mjeru promjene svjetline između susjednih piksela. Rubovi se prepoznaju upravo na onim mjestima gdje je iznos gradijenta najveći, odnosno gdje dolazi do nagle promjene svjetline. Ovaj operator je jednostavan, brz i često se koristi za osnovno prepoznavanje kontura.
- Cannyjev detektor rubova (engl. *Canny edge detector*) - predstavlja napredniji pristup. Prvo zaglađuje sliku radi uklanjanja šuma, zatim računa gradijent, pronalazi potencijalne rubove i na kraju filtrira samo one najizraženije koristeći histerezne pragove (engl. *hysteresis thresholds*). Poznat je po visokoj preciznosti i robusnosti u detekciji tankih, jasnih rubova.
- Harrisov detektor značajki (engl. *Harris corner detector*) - identificira karakteristične točke, najčešće tzv. kutove (engl. *corners*) ili sjecišta dvaju rubova. Kutovi su posebno korisni jer predstavljaju mjesta u slici gdje se intenzitet svjetline mijenja u više smjerova istovremeno, pa ih je lako razlikovati i pouzdano prepoznati čak i ako se slika blago rotira, skalira ili je djelomično zaklonjena.

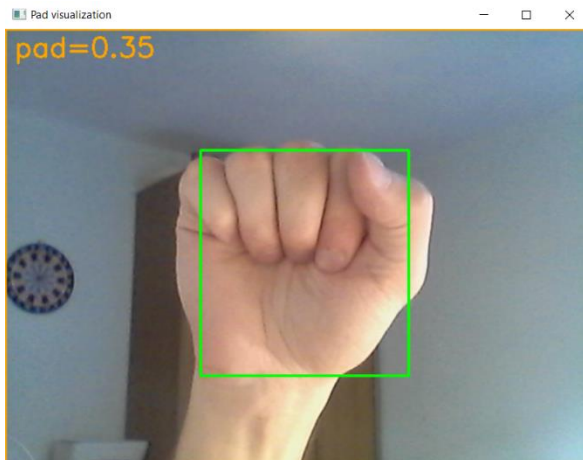
Detekcija rubova i značajki omogućuje izdvajanje ključnih elemenata ruke i prstiju, što je temelj za pouzdano prepoznavanje i analiziranje različitih gesta.

Provedbom navedenih koraka obrade, slika postaje tehnički ispravna, standardizirana i optimalno pripremljena za daljnju analizu. Ovakvom obradom izdvajaju se i naglašavaju ključne informacije o obliku i strukturi ruke, što osigurava kvalitetan temelj za sljedeće korake: detekciju i praćenje ruke. U toj fazi koriste se tehnike računalnog vida koje omogućuju automatsku identifikaciju i praćenje gesta ruke u stvarnom vremenu. Upravo računalnom vidu i procesima detekcije i praćenja posvećeno je sljedeće poglavlje.

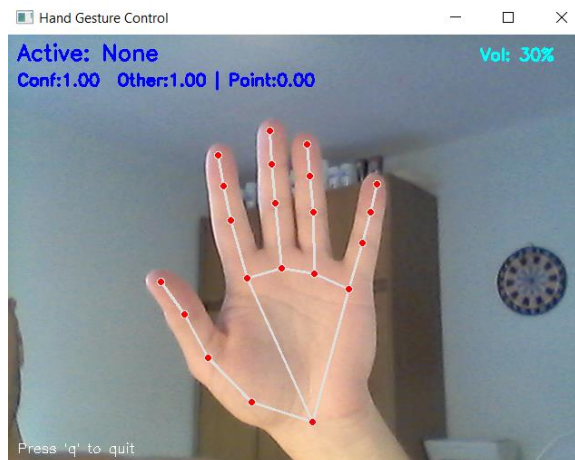
### **3.5. Definicija i svrha računalnog vida**

Računalni vid (engl. *computer vision*) je interdisciplinarno područje koje objedinjuje računarstvo, matematiku, elektrotehniku i kognitivne znanosti, a bavi se razvojem metoda i algoritama za automatsko stjecanje, obradu, analizu i interpretaciju vizualnih informacija iz slika i videozapisa. Cilj računalnog vida je omogućiti računalima ili sustavima s ugrađenim računalima da “vide” tj. prepoznaju i razumiju objekte, scene i događaje iz svoje okoline na način koji je usporediv s ljudskom percepcijom. Prema Szeliskom: „Računalni vid nastoji razviti algoritme koji omogućuju računalima da iz digitalnih slika i videozapisa izvuku semantički korisne informacije, nalik onome što čini ljudski vizualni sustav.” [13]. Rezultat takve obrade mogu biti prepoznati objekti (npr. detekcija ruke na slici 3.9.), detektirani pokreti (npr. detekcija pokreta na slici 3.10) , analizirane scene ili interpretirani složeniji obrasci ponašanja na temelju vizualnih podataka.

Svrha računalnog vida je omogućiti strojno razumijevanje svijeta kroz slike, čime se otvara mogućnost automatizacije zadataka koji zahtijevaju vizualnu percepciju. U praksi, računalni vid nalazi široku primjenu: od prepoznavanja lica i biometrije, medicinske dijagnostike, autonomnih vozila, do sigurnosnih sustava i pametnih gradova. Jedan od najdinamičnijih smjerova razvoja računalnog vida je upravo interaktivno upravljanje uređajima putem gesta ruke.

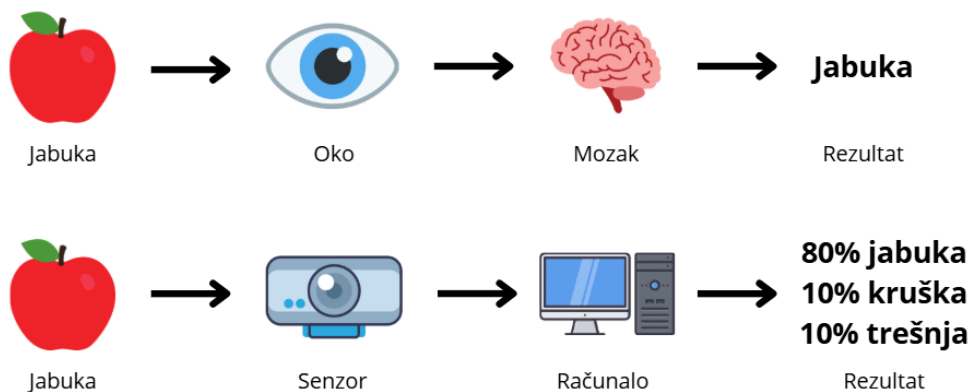


Slika 3.9. Detekcija ruke



Slika 3.10. Detekcija pokreta

Funkcionalno, računalni vid nastoji replicirati ili nadopuniti sposobnosti ljudskog oka i mozga u razumijevanju vizualnog svijeta, kao što se vidi na slici 3.11. Ljudsko oko kontinuirano prima ogroman broj vizualnih informacija koje mozak automatski obrađuje, prepoznaje i interpretira, omogućujući trenutačnu identifikaciju objekata, prepoznavanje pokreta i donošenje odluka u stvarnom vremenu. Slično tome, računalni vid koristi senzore (npr. kamere) za hvatanje slika i napredne algoritme za analizu i interpretaciju tih podataka. Iako je tehnologija računalnog vida u nekim zadacima dosegla ili čak nadmašila ljudsku preciznost (primjerice u detekciji detalja na slikama visoke rezolucije), još uvijek su izazovi poput razumijevanja složenih scena, generalizacije ili prepoznavanja u neidealnim uvjetima područja gdje je ljudski vid superioran [14].



Slika 3.11. Ljudska i računalna percepcija [14]

Kako je prethodno opisano, geste ruke predstavljaju univerzalni i intuitivan oblik neverbalne komunikacije koji je, zahvaljujući napretku digitalnih tehnologija, postao prirodno sučelje za interakciju s računalima i pametnim uređajima. Računalni vid čini temeljnu tehnologiju u razvoju sustava za prepoznavanje gesta ruke. Upravo računalni vid omogućuje automatsku detekciju, praćenje i analizu pokreta ruke, čime se geste pretvaraju u jasno definirane naredbe za digitalne sustave.

Povezivanjem računalnog vida s gestualnim upravljanjem otvaraju se brojne mogućnosti primjene, kako u svakodnevnim interakcijama (npr. već spomenuto upravljanje pametnim telefonom ili kamerom gestom ruke), tako i u naprednim aplikacijama proširene stvarnosti te interaktivnim sustavima za osobe s invaliditetom. Računalni vid omogućuje interpretaciju i statičkih i dinamičkih gesti, razlikujući položaj i konfiguraciju prstiju u jednom trenutku, ali i složene sekvence pokreta kroz vrijeme. Sustavi koji se temelje na računalnom vidu danas mogu prepoznavati jednostavne naredbe, ali i vrlo kompleksne izraze i obrasce ponašanja, čime se svakodnevna interakcija između čovjeka i stroja podiže na novu razinu.

Napredak u računalnom vidu doveo je do razvoja robusnih sustava za prepoznavanje gesta ruke koji su sposobni raditi u stvarnom vremenu, prepoznajući geste i u zahtjevnim uvjetima, poput promjenjivog osvjetljenja, raznolikih pozadina ili različitih korisnika. Time računalni vid postaje ključna karika u stvaranju prirodnog sučelja između čovjeka i tehnologije, omogućujući beskontaktno, prilagodljivo i sofisticirano upravljanje.

### **3.6. Uloga računalnog vida u prepoznavanju gesta ruke**

U sustavima za prepoznavanje gesta ruke, računalni vid ima ključnu ulogu u inicijalnoj obradi vizualnih podataka, odnosno u fazama detekcije i praćenja ruke. Iako se cjelokupni proces od prve slike do prepoznate geste može sagledavati kao šira domena računalnog vida, u ovom radu će se jasnije razdvojiti odgovornosti pojedinih komponenti. Detekcija i praćenje ruke bit će analizirani kao temeljne funkcije računalnog vida, dok se faza prepoznavanja gesta, zbog svoje kompleksnosti i primjene naprednih modela, prepušta algoritmima dubokog učenja. Takva podjela omogućuje jasnije razumijevanje uloge i specifičnih izazova svake faze, premda je u praksi granica između njih često fluidna.

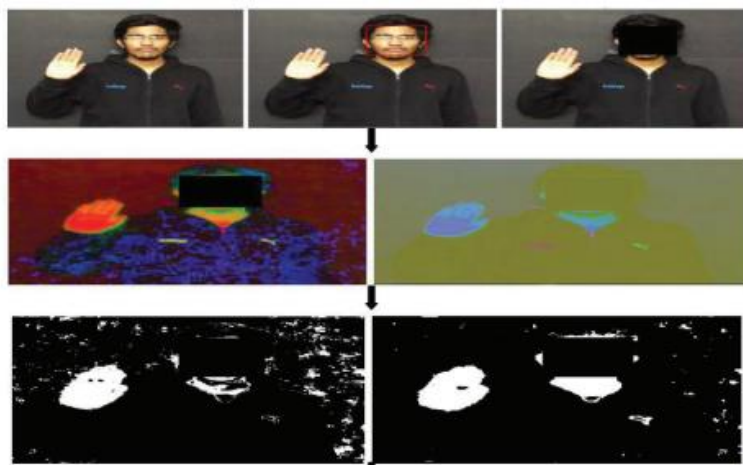
### 3.6.1. Detekcija

Detekcija geste predstavlja prvu i temeljnu fazu u sustavu za prepoznavanje gesta ruke koji se temelji na računalnom vidu. Cilj detekcije je precizno locirati i izdvojiti ruku iz ulazne slike ili videozapisa, čime se omogućuje daljnja analiza njezinih pokreta. Najčešće se provodi uporabom metoda segmentacije slike, pri čemu se ruka izdvaja na temelju karakterističnih značajki, poput boje kože, oblika, kontura ili uz pomoć markera. Segmentacija može biti izazovna zbog raznolikosti pozadina, promjena osvjetljenja i varijacija u izgledu ruke, osobito kod metoda koje se oslanjaju na boju kože [7].

Segmentacija prema boji kože jedna je od najčešće korištenih tehnika segmentacije. Ova metoda koristi činjenicu da ljudska koža ima relativno uzak raspon vrijednosti u određenim prostorima boja, što omogućuje izdvajanje piksela koji odgovaraju boji kože iz slike [15].

Prvi korak u ovom procesu obično je pretvorba slike iz standardnog RGB prostora boja u prikladniji prostor, poput HSV ili YCbCr prostora, gdje su informacije o boji i svjetlini jasnije razdvojene. U tim prostorima, određuju se rasponi vrijednosti (engl. *thresholds*) koji odgovaraju različitim tonovima kože. Zatim se pragovnim izdvajanjem (engl. *thresholding*) određuje binarna maska kojom se na slici izdvajaju svi pikseli unutar definiranih raspona vrijednosti, dok se preostali pikseli zanemaruju ili postavljaju na nulu. Prednost ove metode je njezina brzina i jednostavnost, no osjetljiva je na promjene osvjetljenja, zasjenjenje te prisutnost objekata slične boje u pozadini.

Na slici 3.12. vizualizirana je navedena metoda. Slika se najprije pretvara iz RGB u HSV (lijevo) i YCbCr prostor boja (desno). Zatim se za svaki prostor određuje odgovarajući raspon vrijednosti koji karakterizira boju kože u tom prostoru. Nakon toga, primjenjuje se postupak pragovnog izdvajanja, pri čemu se na temelju definiranih raspona generiraju binarne maske (crno-bijele). Dobivene maske kasnije se koriste tako da se primjenjuju na izvornu sliku, čime se iz slike izdvaja samo regija ruke, dok se ostatak slike zanemaruje.



*Slika 3.12. Segmentacija prema boji kože [16]*

Drugi često korišten pristup temelji se na segmentaciji prema obliku ruke, pri čemu se naglasak stavlja na prepoznavanje karakterističnog oblika odnosno konture ruke unutar slike. Proces najčešće započinje binarizacijom, tijekom koje se izvorna slika pretvara u binarnu, odnosno crno-bijelu sliku, na temelju unaprijed definiranih kriterija kao što su boja kože, svjetlina ili drugi relevantni parametri. Ovim postupkom izdvajaju se pikseli koji pripadaju potencijalnoj regiji ruke, dok se svi ostali pikseli označavaju kao pozadina.

Nakon binarizacije, slijedi primjena algoritama za detekciju rubova, pri čemu se najčešće koristi Cannyjev detektor rubova, koji omogućuje precizno prepoznavanje prijelaza između ruke i pozadine na slici. Rezultat ovog koraka su konture, odnosno zatvorene linije koje omeđuju povezane skupove piksela iste vrijednosti. U sljedećoj fazi provodi se analiza svih pronađenih kontura s ciljem identificiranja one koja najviše odgovara očekivanoj geometriji ruke. Najčešće se odabire kontura s najvećom površinom ili duljinom, budući da ruka obično zauzima dominantan dio segmentirane regije, dok se manje ili nepravilne konture zanemaruju kao šum. Osim toga, u nekim pristupima dodatno se analiziraju i drugi geometrijski pokazatelji, poput oblika ili omjera stranica konture, kako bi se dodatno povećala pouzdanost detekcije [17]. Ovakav pristup pokazuje veću otpornost na varijacije u boji kože u odnosu na metode temeljene isključivo na boji, ali može biti osjetljiv na složene pozadine ili situacije u kojima se ruka preklapa s drugim objektima na slici.

Na slici 3.13. segmentacija prema boji kože služi kao pripremni korak za izdvajanje regije od interesa, čime se smanjuje kompleksnost pozadine i omogućuje fokusiranja primjena algoritama

za detekciju kontura, poput Cannyjevog detektora rubova. Međutim, jasno je vidljivo da prisutnost objekata sa sličnim kolorističkim svojstvima u pozadini negativno utječe na pouzdanost segmentacije.



Slika 3.13. Segmentacija po obliku

Za poboljšanje rezultata segmentacije, često se primjenjuju morfološke operacije. Morfološke operacije su skup tehnika iz matematičke morfologije koje se primjenjuju na binarne slike kako bi se uklonile nepravilnosti, popunile rupe ili povezali diskontinuirani dijelovi regije ruke.

Najčešće operacije:

- Erozija - smanjuje veličinu objekta uklanjanjem rubnih piksela.
- Dilatacija - proširuje objekt dodavanjem piksela na njegove rubove.
- Otvaranje - sastoji se od erozije praćene dilatacijom; koristi se za uklanjanje malih šumova.
- Zatvaranje - sastoji se od dilatacije praćene erozijom; koristi se za popunjavanje malih rupa unutar objekta.

Kombiniranjem ovih operacija, segmentirana regija ruke postaje glađa i prikladnija za daljnje analiziranje gesta [12].

Dobar rezultat detekcije osigurava kvalitetan ulaz za preostale faze obrade, a greške u ovoj fazi mogu dovesti do gubitka informacija i otežati kasnije praćenje ili prepoznavanje gesti. Stoga se u suvremenim sustavima sve češće koriste modeli dubokog učenja, osobito konvolucijske neuronske mreže, za automatsko prepoznavanje i izdvajanje ruke čak i u nepovoljnim uvjetima. Za specifične zadatke često se primjenjuje tehnika *transfer learninga*, što omogućuje robusnu i brzu detekciju ruke bez potrebe za ručnim podešavanjem parametara, čak i u složenim scenama. Jedan od



najpoznatijih alata ove vrste je Google MediaPipe, koji omogućuje pouzdanu detekciju i praćenje ruke u stvarnom vremenu, čime je postao standard u mnogim modernim aplikacijama za prepoznavanje gesta.

U nekim slučajevima, kao dodatna pomoć detekciji koriste se vizualni markeri ili rukavice s prepoznatljivim uzorcima, što može poboljšati točnost izdvajanja ruke iz složene pozadine. Dodatno, često se primjenjuje fuzija podataka iz više izvora, primjerice kombinacija RGB kamere i dubinske kamere (npr. Microsoft Kinect, Intel RealSense). Integracija dubinskih podataka omogućuje preciznije izdvajanje ruke iz složenih pozadina, osobito u uvjetima s puno vizualnih smetnji ili kod korisnika različite boje kože.

### 3.6.2. Praćenje

Praćenje geste obuhvaća kontinuiranu identifikaciju i praćenje položaja ruke i njenih dijelova kroz niz uzastopnih slika ili kadrova u videozapisu. Glavni cilj ove faze je bilježenje dinamike pokreta, odnosno putanje, brzine i promjena konfiguracije prstiju i dlana tijekom izvođenja geste. U praksi se najčešće koriste tri glavna pristupa praćenju: metode temeljene na obliku i boji, metode temeljene na filtriranju te metode temeljene na detekciji karakterističnih točaka, uključujući skeletne modele.

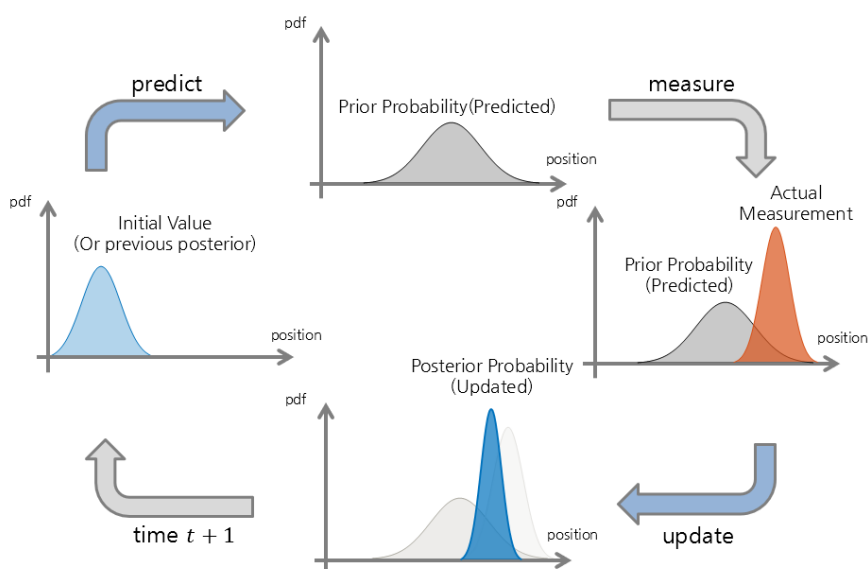
Metode temeljene na obliku i boji koriste analizu vizualnih karakteristika ruke, poput kontura, površine ili specifične boje kože. Praćenjem obrisa ili regije ruke kroz niz kadrova, moguće je prepoznati promjene u položaju i obliku, što je posebno korisno kod složenih ili brzih gesti. Međutim, ove metode mogu biti vrlo osjetljive na varijacije u osvjetljenju ili prisutnost objekata slične boje u pozadini, zbog čega se u suvremenim sustavima koriste sve rjeđe kao glavna metoda praćenja. Danas se one najčešće primjenjuju kao pomoćni ili inicijalni korak za grubo izdvajanje ruke, kao što je već objašnjeno, dok se za precizno i robusno praćenje koriste naprednije metode, poput filtriranja ili detekcije karakterističnih točaka i skeletnih modela.

Metode temeljene na filtriranju koriste matematičke modele za predviđanje i ispravljanje položaja ruke na temelju prethodnih mjerenja. Najpoznatiji su Kalmanov filter i čestični filter (engl. *particle filter*).

Kalmanov filtar je optimalni linearni filtar koji se koristi za procjenu stanja dinamičkog sustava iz niza mjerenja koja su opterećena šumom. U kontekstu praćenja ruke, Kalmanov filtar modelira položaj i brzinu ruke kao stanja sustava te koristi dva glavna koraka: predviđanje i ažuriranje. U fazi predviđanja, na temelju prethodnog stanja i poznate dinamike sustava, filtar procjenjuje novo stanje ruke.

Nakon što se dobije novo mjerenje, filtar ga koristi za ispravljanje prethodno procijenjenog stanja. Ako je novo mjerenje vrlo pouzdano, filtar će mu pridati veću važnost u određivanju trenutnog položaja ruke. S druge strane, ako je mjerenje manje pouzdano ili sadrži više šuma, filtar će se više oslanjati na predviđanje temeljeno na prethodnim stanjima. Na taj način, Kalmanov filtar dinamički kombinira informacije iz predviđanja i novih mjerenja, kako bi postigao što točniju i stabilniju procjenu trenutnog stanja ruke. Ova iterativna procedura omogućuje filtriranje šuma i glatko praćenje putanje ruke čak i kada su mjerenja djelomično nepouzdana ili nedostupna [18].

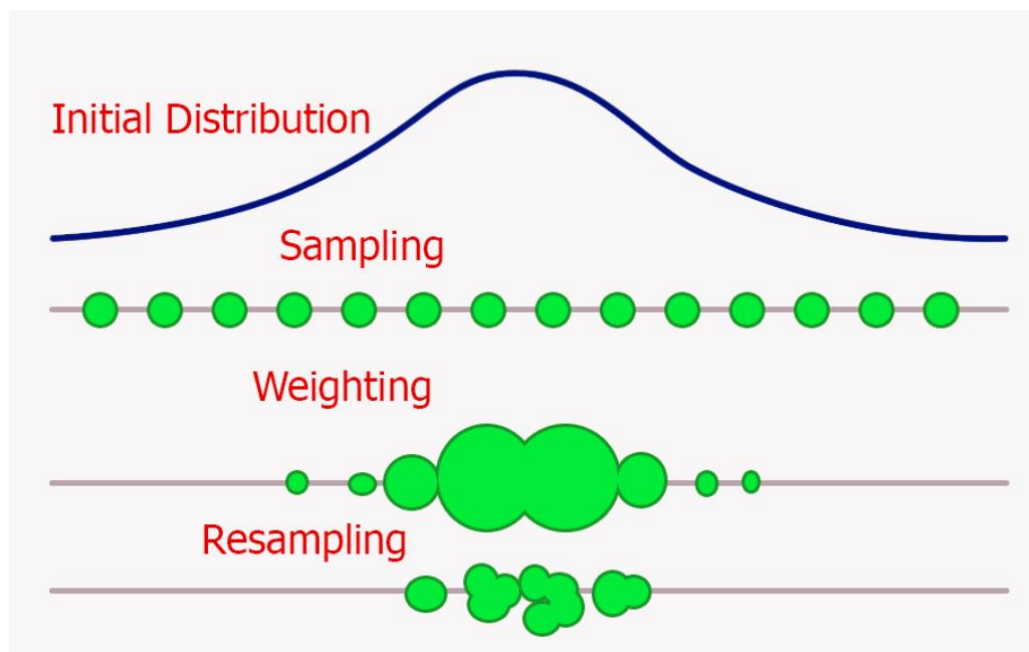
Na slici 3.14. prikazana je pojednostavljena shema Kalmanovog filtra koja uključuje faze predviđanja, mjerenja i ažuriranja. U ovom primjeru, predikcija je vrlo nesigurna i karakterizira je široka distribucija, dok je mjerenje izuzetno pouzdano i ima usku distribuciju. Zbog toga se prilikom ažuriranja procjene stanja Kalmanov filtar u većoj mjeri oslanja na novo mjerenje, što rezultira točnijom i pouzdanijom konačnom procjenom.



Slika 3.14. Pojednostavljeni model Kalmanovog filtra [19]

Čestični filter (eng. *particle filter*) je napredna metoda za procjenu stanja sustava koja nije ograničena na linearne ili normalno distribuirane procese. Umjesto jedne procjene stanja, čestični filter koristi velik broj čestica, pri čemu svaka čestica predstavlja jednu moguću pretpostavku o trenutnom položaju i kretanju ruke. Tijekom svake iteracije, ove čestice se ažuriraju prema modelu kretanja (npr. prema pretpostavljenoj brzini i smjeru ruke). Nakon toga, svaka čestica dobiva određenu težinu (engl. *weighting*) na temelju toga koliko dobro odgovara novom mjerenju. Što se bolje poklapa s podacima iz slike ili senzora, to je njezina težina veća.

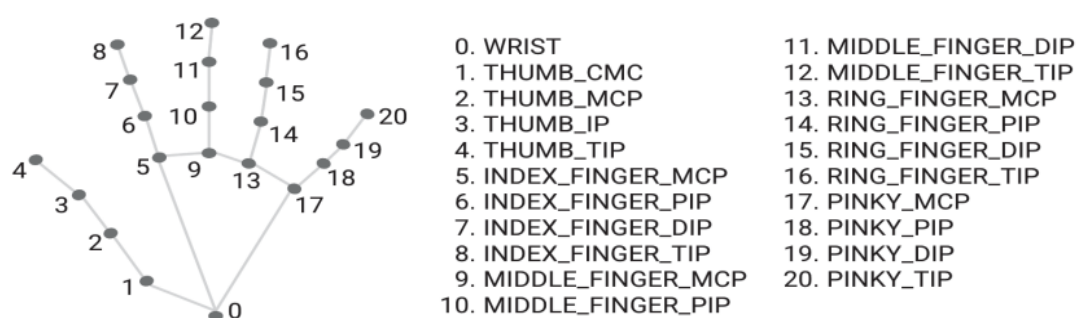
Čestice s malim težinama, odnosno one koje se slabo podudaraju sa stvarnim stanjem ruke, postupno se izbacuju iz daljnjeg praćenja. S druge strane, čestice koje se dobro podudaraju s novim mjerenjima ostaju u sustavu, a njihov broj se povećava tako što se odabiru više puta u sljedećem koraku u procesu ponovnog uzorkovanja (engl. *resampling*) kao što se vidi na slici 3.15. Drugim riječima, umjesto manje vjerojatnih čestica, u sustavu ostaju i češće se biraju one koje najbolje opisuju trenutni položaj ruke. Time filter svoje procjene usmjerava prema najvjerojatnijim mogućim stanjima, što omogućuje precizno i stabilno praćenje ruke čak i u složenim ili šumovitim uvjetima [20].



Slika 3.15. Proces čestičnog filtriranja [20]

Metode temeljene na detekciji karakterističnih točaka i skeletnim modelima podrazumijevaju identifikaciju i praćenje specifičnih točaka na ruci, poput vrhova prstiju, zglobova ili centra dlana. Praćenjem ovih značajki moguće je vrlo precizno rekonstruirati putanju i promjene konfiguracije ruke tijekom izvođenja geste.

Suvremeni skeletni modeli, poput modela na slici 3.16., koriste povezane točke za modeliranje cijele ruke te omogućuju praćenje svih zglobova i prstiju u stvarnom vremenu. Ovaj pristup pruža detaljne informacije o orijentaciji i konfiguraciji ruke te je posebno učinkovit u aplikacijama koje zahtijevaju visoku preciznost i robusnost. Jedan od najpoznatijih sustava ove vrste je MediaPipe, koji omogućuje implementaciju ovakvih modela i služi kao temelj za razvoj brojnih aplikacija za prepoznavanje i praćenje gesta ruke.



*Slika 3.16. MediaPipe skeletni model [21]*

Ove metode omogućuju precizno praćenje čak i kada se ruka djelomično zaklanja ili mijenja orijentaciju u prostoru. Tijekom praćenja važno je osigurati robusnost algoritma na brze pokrete, privremena zaklanjanja ruke i promjene u osvjetljenju. Praćenje se dodatno može poboljšati kombiniranjem više senzora ili izvora podataka (npr. istodobno korištenje RGB i dubinskih kamera), čime se smanjuje mogućnost gubitka praćenja. Rezultat ove faze su vremenske sekvence podataka, poput putanja i gestualnih obrazaca, koje se dalje koriste u procesu prepoznavanja.

## **4. Strojno i duboko učenje**

### **4.1. Motivacija**

Nakon što su u prethodnim poglavljima detaljno obrađene faze detekcije i praćenja ruke u okviru računalnog vida, sljedeća ključna faza u sustavu za prepoznavanje gesta ruke jest faza prepoznavanja. Ova faza predstavlja završni i najvažniji segment obrade, gdje je cilj na temelju podataka dobivenih iz prethodnih koraka ispravno klasificirati i interpretirati gestu te je povezati s unaprijed definiranim skupom komandi ili značenja. Od presudne je važnosti u ovoj fazi izdvajanje značajki, pri čemu se iz informacija o položaju i pokretu ruke izdvajaju oni parametri i obrasci koji najpreciznije opisuju svaku gestu. Kvaliteta i reprezentativnost izdvojenih značajki izravno utječu na uspješnost klasifikacije, kao i na otpornost sustava na varijacije u izvođenju gesta.

Premda su se u ranijim pristupima koristile različite metode za izdvajanje značajki i klasifikaciju gesta, uključujući predloške, statističke klasifikatore i skrivene Markovljeve modele, suvremena istraživanja sve više primjenjuju tehnike dubokog učenja. Posebno se izdvajaju konvolucijske neuronske mreže, koje omogućuju automatsko izdvajanje relevantnih značajki iz vizualnih podataka i prepoznavanje složenih obrazaca gesta s visokom razinom preciznosti. U ovom radu, proces prepoznavanja gesta bit će detaljno prikazan upravo kroz primjenu konvolucijskih neuronskih mreža, s posebnim naglaskom na njihove prednosti i mogućnosti u kontekstu automatskog prepoznavanja gesta ruke.

Kako bi se u potpunosti razumjelo funkcioniranje konvolucijskih neuronskih mreža i njihovu primjenu u prepoznavanju gesta, nužno je prethodno steći temeljno razumijevanje principa strojnog i dubokog učenja. Osnovni pojmovi iz ovih područja, poput modela učenja, neuronskih mreža, procesa treniranja i optimizacije, predstavljaju polazište za razumijevanje naprednih arhitektura i metoda koje čine temelj suvremenih sustava za automatsko prepoznavanje gesta ruke.

## 4.2. Strojno učenje

Strojno učenje (engl. *machine learning*) predstavlja područje umjetne inteligencije koje se bavi razvojem algoritama i modela koji omogućuju računalnim sustavima da automatski uče iz podataka i poboljšavaju svoje performanse bez potrebe za eksplicitnim programiranjem svih koraka. Temeljna svrha strojnog učenja je omogućiti računalima da na temelju iskustva, odnosno analize postojećih podataka, prepoznaju obrasce, donose zaključke i predviđaju buduće ishode ili ponašanja. Time se strojno učenje ističe kao ključna tehnologija za automatizaciju rješavanja složenih zadataka, koji su ranije zahtijevali ljudsko znanje, iskustvo i procjenu. U suvremenim računalnim sustavima strojno učenje nalazi široku primjenu, od prepoznavanja govora i slike, preko medicinske dijagnostike, do financijskih predviđanja i autonomnih vozila [22].

### 4.2.1. Vrste strojnog učenja

Strojno učenje obuhvaća nekoliko temeljnih pristupa, koji se razlikuju prema načinu na koji algoritmi uče iz dostupnih podataka. Najrašireniji i najčešće primjenjivan oblik je nadzirano učenje (engl. *supervised learning*). U ovom pristupu, modelu su na raspolaganju podaci kod kojih su ulazne vrijednosti uparene s točno poznatim izlazima, odnosno oznakama (engl. *labels*). Model u procesu treniranja uči povezivati ulazne podatke s pripadajućim izlazima, s ciljem izgradnje općeg pravila koje će omogućiti precizna predviđanja na novim, do tada neviđenim podacima. Nadzirano učenje je temelj brojnih praktičnih primjena, osobito u problemima klasifikacije, gdje je zadaća modela svrstati ulaz u jednu od unaprijed definiranih kategorija, te u regresijskim zadacima, gdje model predviđa kontinuirane numeričke vrijednosti.

Ključni element nadziranog učenja je prisutnost nadzornika, odnosno procesa u kojem stručnjak ili automatizirani sustav mora unaprijed označiti (engl. *labeling*) sirove podatke (engl. *raw data*). Ovaj proces označavanja podrazumijeva pridruživanje svake ulazne instance pripadajućoj izlaznoj vrijednosti ili klasi, čime se stvara strukturirani skup podataka na kojem model može učiti. Na slici 4.1. prikazan je proces označavanja sirovih podataka tj. označavanja pripadnosti crvenoj ili plavoj klasi. Kvaliteta i točnost oznaka izravno utječu na učinkovitost i pouzdanost konačnog modela, budući da pogrešno označeni podaci mogu dovesti do učenja neželjenih obrazaca.

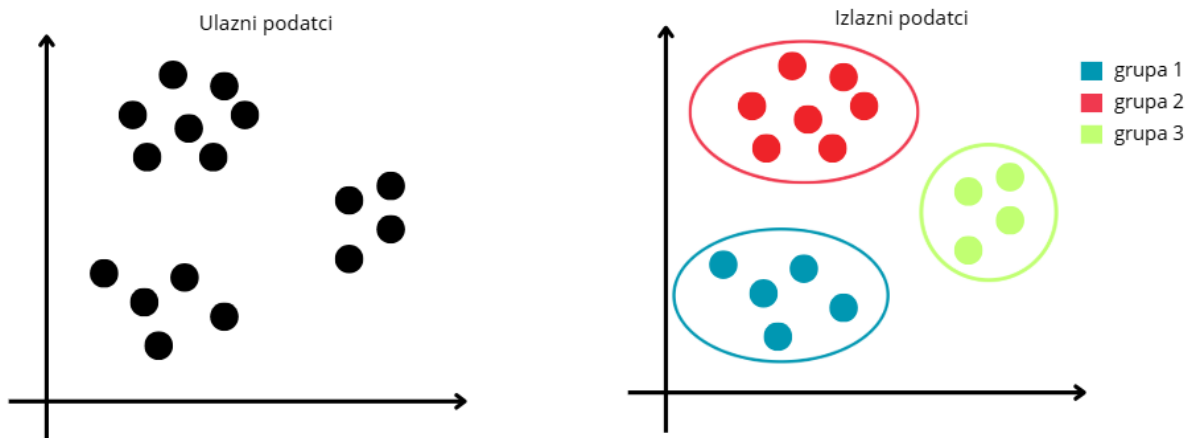


*Slika 4.1. Proces označavanja podataka*

Označavanje podataka često je vremenski zahtjevan i resursno intenzivan proces, posebice kod velikih skupova podataka, ali je nužan kako bi se omogućilo učinkovito i točno učenje modela u okviru nadziranog učenja.

Za razliku od nadziranog učenja, nenadzirano učenje (engl. *unsupervised learning*) ne raspolaže unaprijed poznatim izlaznim oznakama. Algoritmi nenadziranog učenja usmjereni su na analizu i istraživanje samih podataka, s ciljem otkrivanja skrivenih obrazaca, struktura ili odnosa među podatkovnim instancama. Budući da nema informacija o ispravnim rješenjima, model samostalno identificira sličnosti, razlike i moguće grupe unutar skupa podataka. Ovaj pristup posebno je vrijedan u situacijama kada predznanje o mogućim izlazima nije dostupno ili kada postoji potreba za segmentacijom podataka radi boljeg razumijevanja njihovih svojstava. Najčešća primjena nenadziranog učenja je klasteriranje (engl. *clustering*), odnosno grupiranje podataka na temelju međusobne sličnosti. Nenadzirano učenje često se koristi u otkrivanju anomalija, analizi ponašanja korisnika, organizaciji velike količine neoznačenih podataka i kao pripremni korak u složenijim analitičkim postupcima.

Na slici 4.2. prikazan je proces klasteriranja. Na lijevoj strani vidimo skup točaka koje nisu unaprijed označene i predstavljaju sirove podatke. Na desnoj strani te točke su podijeljene u tri različite grupe (klasterne) obojene plavom, crvenom i zelenom bojom. To znači da je algoritam klasteriranja pronašao sličnosti među podacima i grupirao ih prema određenom kriteriju, primjerice prema međusobnoj udaljenosti točaka u prostoru.



Slika 4.2. Proces klasteriranja podataka

Treći temeljni oblik strojnog učenja je učenje potkrepljivanjem (engl. *reinforcement learning*), koje se temelji na konceptu interakcije inteligentnog agenta s dinamičkim okruženjem. Agent tijekom vremena donosi niz uzastopnih odluka, dok mu okruženje daje povratnu informaciju u obliku nagrade ili kazne, ovisno o kvaliteti odabrane akcije u odnosu na zadani cilj. Cilj agenta je razviti strategiju ponašanja koja maksimizira ukupnu nagradu kroz vrijeme. Ovaj pristup posebno je pogodan za probleme kojima je karakteristična potreba za donošenjem sekvencijalnih odluka, poput automatiziranih sustava upravljanja, robotske navigacije ili učenja igranja složenih igara.

#### 4.2.2. Priprema podataka

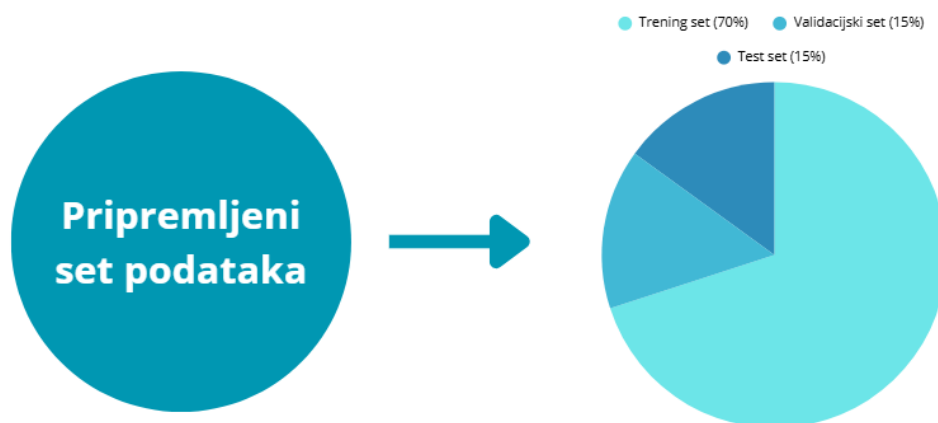
Prva i temeljna faza svakog projekta strojnog učenja uključuje pažljivo prikupljanje relevantnih podataka koji adekvatno reprezentiraju problem koji se želi riješiti. Podaci mogu biti prikupljeni iz različitih izvora, kao što su baze podataka, senzori, internetske usluge, ankete, ili generirani eksperimentalno. Osim dovoljne količine podataka, izuzetno je važna njihova kvaliteta; podaci moraju biti reprezentativni, raznoliki i sadržavati dovoljno informacija da model može identificirati značajne obrasce. Nedostatak ili nepravilna raspodjela podataka često dovodi do problema loše generalizacije modela. U takvoj situaciji model nauči specifičnosti i slučajne obrasce unutar dostupnog skupa podataka, što mu otežava prepoznavanje i točno predviđanje na novim, neviđenim podacima.



Nakon prikupljanja, podaci najčešće zahtijevaju opsežnu pripremu prije nego što postanu prikladni za unos u modele strojnog učenja. Ova faza obuhvaća različite postupke čišćenja, poput uklanjanja pogrešaka, zamjene ili popunjavanja nedostajućih vrijednosti, otkrivanja i ispravljanja anomalija te rješavanja problema neuravnoteženih klasa.

Ključno je razumjeti da je ulazni podatak u strojnome učenju najčešće predstavljen kao vektor koji se sastoji od više vrijednosti, pri čemu svaka od njih predstavlja određeni atribut ili značajku (engl. *feature*). Atributi opisuju specifične karakteristike ili mjerene veličine promatranog objekta, primjerice visinu, težinu, dob, boju ili rezultat nekog mjerenja, i mogu biti numerički ili kategorički. Pravilna obrada i priprema tih atributa izravno utječu na uspješnost modela strojnog učenja. Zbog toga se podaci nakon čišćenja često normaliziraju ili standardiziraju kako bi svi atributi imali usporedive vrijednosti, što poboljšava učinkovitost i stabilnost mnogih algoritama.

Na kraju, pripremljeni podaci dijele se na skup za treniranje (engl. *training set*), skup za validaciju (engl. *validation set*) i skup za testiranje (engl. *test set*). Ova podjela omogućuje objektivnu procjenu performansi razvijenog modela na podacima koji nisu korišteni tijekom procesa učenja, čime se osigurava pouzdanost i generalizacija modela. Važno je napomenuti da omjer podjele ovih skupova nije strogo definiran, već se njime često eksperimentira kako bi se pronašla najprikladnija raspodjela za konkretan skup podataka i problem koji se analizira. Na slici 4.3. se može vidjeti podjela pripremljenog seta podataka u omjeru 70:15:15.



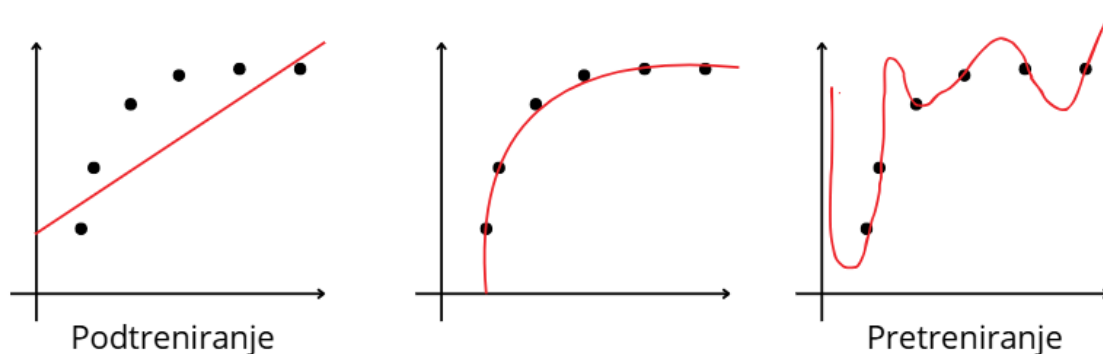
Slika 4.3. Podjela podataka

#### 4.2.3. Treniranje i evaluacija modela

S tako pripremljenim trening skupom podataka, prije početka procesa treniranja potrebno je odabrati odgovarajući model ili algoritam strojnog učenja, što predstavlja ključan korak za uspješno rješavanje zadanog problema. Ovisno o vrsti zadatka, primjerice klasifikaciji ili regresiji, te karakteristikama dostupnih podataka, u obzir dolaze različiti modeli. Među jednostavnijim pristupima često se koriste linearna regresija za predviđanje kontinuiranih vrijednosti i logistička regresija za klasifikacijske zadatke. Kod složenijih problema, primjenu nalaze stabla odlučivanja, koja omogućuju donošenje odluka na temelju hijerarhije uvjeta, te neuronske mreže, koje su iznimno učinkovite u prepoznavanju složenih obrazaca i nelinearnih odnosa u podacima. U praksi se često ispituje više različitih modela kako bi se utvrdilo koji najbolje odgovara konkretnom problemu.

Treniranje modela je proces tijekom kojeg se odabrani model izlaže trening skupu podataka i postupno prilagođava svoje interne parametre (npr. težine u neuronskim mrežama) kako bi minimizirao razliku između predviđenih vrijednosti i stvarnih izlaza. Ta razlika se mjeri pomoću funkcije koštanja (engl. *cost function* ili *loss function*), koja kvantificira pogrešku modela i služi kao vodič za optimizaciju parametara tijekom procesa učenja. Ovaj proces se provodi iterativno, kroz više epoha, koristeći optimizacijske algoritme poput gradijentnog spusta (engl. *gradient descent*). Pravilno treniranje modela uključuje praćenje i izbjegavanje pretreniranja (engl. *overfitting*) i podtreniranja (engl. *underfitting*), što se često postiže tehnikama kao što su regularizacija, rani prekid (engl. *early stopping*), prilagođavanje hiperparametara i slično.

Pretreniranje označava situaciju u kojoj je model previše prilagođen specifičnostima trening skupa, dok podtreniranje označava nedovoljnu prilagođenost modela podacima. Oba problema dovode do već spomenute loše generalizacije, odnosno smanjene sposobnosti modela da točno predviđa rezultate na novim, neviđenim podacima. Na slici 4.4., podtreniranje (lijevi graf) je vidljivo po tome što model, primjerice jednostavna ravna linija, ne uspijeva obuhvatiti stvaran odnos među podacima i ne prati njihov obrazac. S druge strane, pretreniranje (desni graf) prikazano je modelom koji previše precizno prati svaki podatak iz trening skupa, zbog čega unos novog, neviđenog podatka često rezultira netočnim predviđanjem koje ne odražava opći trend u podacima.



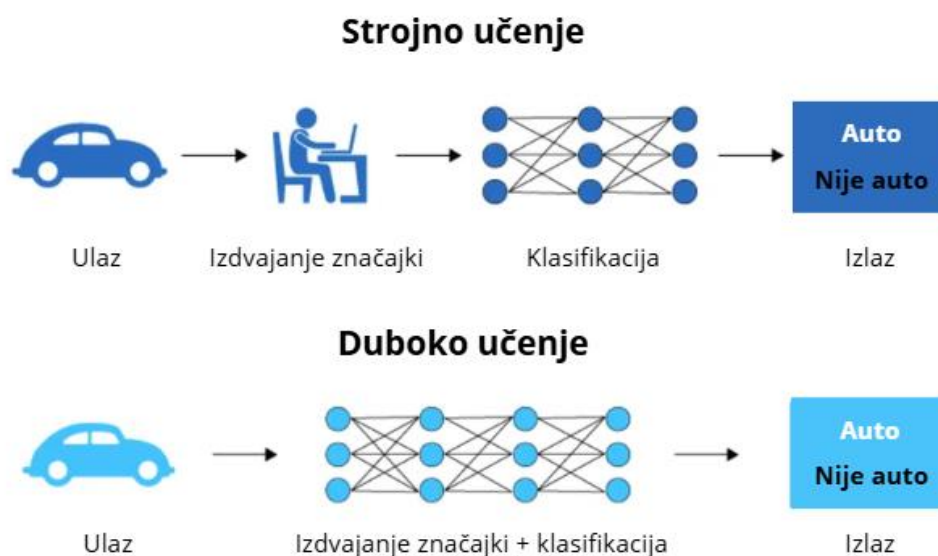
Slika 4.4. Primjer podtreniranog i pretreniranog modela

Nakon što je model istreniran, potrebno je objektivno procijeniti njegovu učinkovitost na podacima koji nisu korišteni tijekom procesa učenja, odnosno na validacijskom i testnom skupu. Evaluacija se najčešće provodi na temelju točnosti ili neke druge prikladne metrike, ovisno o vrsti problema. Dok se validacijski skup koristi za praćenje i podešavanje parametara modela tijekom razvoja, testni skup služi kao konačna provjera te daje objektivnu ocjenu sposobnosti modela da generalizira na potpuno novim podacima. Ova faza omogućuje pravovremeno otkrivanje potencijalnih problema i daje smjernice za daljnje poboljšanje modela, bilo kroz doradu podataka, odabir drugačijeg modela ili optimizaciju njegovih parametara.

### 4.3. Duboko učenje

Duboko učenje (engl. *deep learning*) je metoda strojnog učenja koja koristi višeslojne umjetne neuronske mreže za modeliranje visoko nelinearnih i složenih odnosa u podacima. Ključna prednost dubokog učenja u odnosu na klasične metode strojnog učenja leži u sposobnosti modela da automatski iz sirovih podataka uči reprezentacije, odnosno značajke na različitim razinama apstrakcije. Svaki sloj umjetne neuronske mreže automatski uči apstraktnije i sve složenije karakteristike ulaznih podataka. Na primjer, kod prepoznavanja slike, prvi slojevi mogu otkriti jednostavne obrasce kao što su rubovi ili boje, dok dublji slojevi prepoznaju složenije strukture, oblike ili cijele objekte [23].

U tradicionalnom pristupu, kvaliteta modela uvelike ovisi o ručnoj selekciji i inženjeringu značajki, što zahtijeva domensko znanje i često ograničava primjenjivost modela na složene ili nestrukturirane podatke (poput slika ili zvuka). Duboko učenje, s druge strane, omogućuje modelima da izravno iz podataka uče potrebne karakteristike kao što se vidi na slici 4.5. Time se smanjuje potreba za ručnom intervencijom i otvara mogućnost rješavanja zadataka kao što su prepoznavanje objekata u slikama, obrada prirodnog jezika, analiza govora i mnogi drugi izazovi moderne umjetne inteligencije. Napredak računalne snage, dostupnost velikih skupova podataka i razvoj optimizacijskih algoritama dodatno su ubrzali širenje i uspješnost dubokog učenja u raznim znanstvenim i industrijskim područjima.



*Slika 4.5. Razlika strojnog i dubokog učenja [24]*

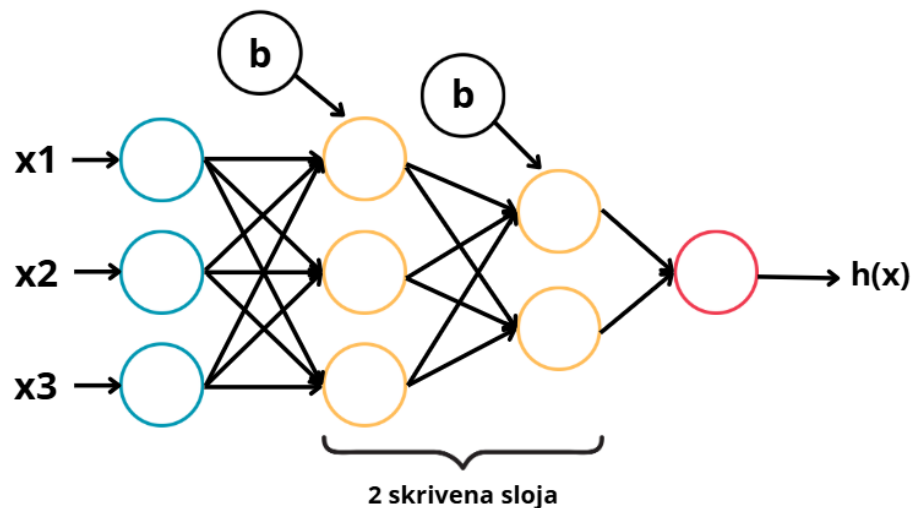
#### 4.3.1. Koncept umjetne neuronske mreže

Umjetna neuronska mreža je matematički model inspiriran strukturom i funkcijom bioloških neurona. Sastoji se od međusobno povezanih jedinica, tzv. umjetnih neurona, organiziranih u slojeve.

Najčešća arhitektura uključuje tri osnovne vrste slojeva:

- Ulazni sloj - prima podatke iz okoline ili iz skupa podataka.
- Skriveni slojevi - jedan ili više slojeva u kojima se odvija većina obrade i transformacije informacija. Svaki neuron u skrivenom sloju prima ulazne vrijednosti, izračunava njihov zbroj pomnožen s odgovarajućim težinama, dodaje vrijednost pomaka (engl. *bias*) i zatim rezultat prosljeđuje kroz nelinearnu aktivacijsku funkciju.
- Izlazni sloj - generira krajnju predikciju ili klasifikaciju modela.

Na slici 4.6. prikazan je model umjetne neuronske mreže koja se sastoji od ukupno četiri sloja. Prvi sloj predstavlja ulazni sloj i sadrži tri neurona koji direktno odgovaraju trima atributima podataka. Svaki od ta tri ulaza samo prenosi vrijednost svog atributa ka narednom sloju. Zatim slijede dva skrivena sloja (prvi s 3 neurona, drugi s 2). Svaki njihov neuron prima sve ulaze iz prethodnog sloja, na svaki ulaz primjenjuje odgovarajuću težinu, dodaje vrijednost pomaka te rezultat zatim prosljeđuje kroz nelinearnu aktivacijsku funkciju, čime mreža uči kombinacije i apstrakcije ulaznih atributa. Nakon skrivenih slojeva nalazi se izlazni sloj s jednim neuronom koji generira krajnju predikciju ili klasifikaciju modela.



Slika 4.6. Model umjetne neuronske mreže

Ključni element umjetne neuronske mreže je aktivacijska funkcija, koja unosi nelinearnost u model i omogućuje mu učenje složenih odnosa. Najčešće korištene aktivacijske funkcije su: sigmoidna funkcija, tanh (tangens hiperbolični) funkcija i ReLU (engl. *Rectified Linear Unit*) funkcija [22].

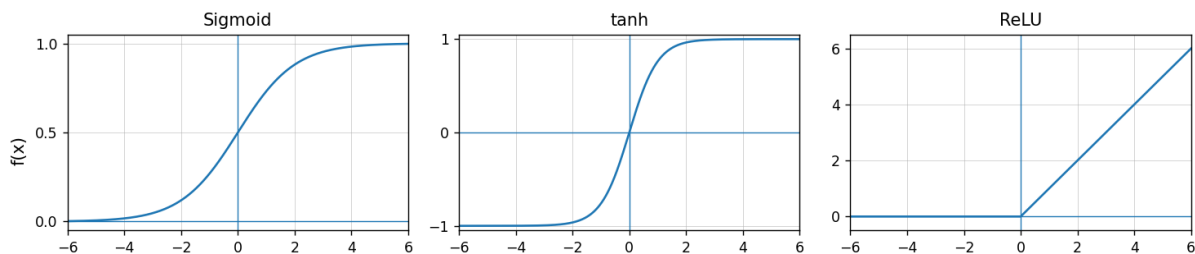
Sigmoidna aktivacijska funkcija (slika 4.7. lijevo) preslikava svaku realnu vrijednost u interval (0, 1). Najčešće se koristi u izlaznim slojevima neuronskih mreža za binarnu klasifikaciju, gdje rezultat funkcije može biti interpretiran kao vjerojatnost pripadnosti jednoj od klasa. Glavno ograničenje sigmoidne funkcije je sklonost pojavi problema nestajućeg gradijenta (engl. *vanishing gradient*), osobito u dubokim mrežama, zbog vrlo malih derivacija za ekstremne ulazne vrijednosti. Definirana je izrazom:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (4.1)$$

Tanh aktivacijska funkcija (slika 4.7. sredina) preslikava svaku realnu vrijednost u interval (-1, 1), a funkcija je centrirana oko nule, što može pridonijeti bržoj konvergenciji modela tijekom treniranja. Unatoč toj prednosti, tanh funkcija također pati od problema nestajućeg gradijenta pri većim apsolutnim vrijednostima ulaza. Definirana je izrazom:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (4.2)$$

ReLU aktivacijska funkcija (slika 4.7. desno) radi tako da za svaku ulaznu vrijednost vraća nulu ako je ulaz manji od nule, a samu tu vrijednost ako je ulaz pozitivan. Izlazni interval ove funkcije je od nule do beskonačnosti, uključujući nulu. Vrlo je popularna zbog jednostavne implementacije i učinkovitosti. Omogućuje brže i stabilnije treniranje dubokih mreža jer ublažava problem nestajućeg gradijenta za pozitivne ulazne vrijednosti. Međutim, može doći do problema tzv. „mrtvih neurona“ ako neuroni tijekom treniranja stalno primaju negativne ulaze, zbog čega njihova aktivacija ostaje trajno nula.



Slika 4.7. Prikaz aktivacijskih funkcija

#### 4.3.2. Proces treniranja umjetne neuronske mreže

Treniranje umjetne neuronske mreže proces je u kojem model uči optimalne vrijednosti svojih parametara kako bi što bolje mapirao ulazne podatke na željene izlaze [22]. Ovaj proces uključuje nekoliko ključnih koraka:

- Propagacija unaprijed (engl. *forward propagation*) - prolazak podataka od ulaznog sloja prema izlaznom sloju, pri čemu se na svakom sloju računaju linearne kombinacije ulaznih i pripadajućih težinskih vrijednosti, a na dobiveni rezultat primjenjuju se aktivacijske funkcije. Rezultat na izlaznom sloju predstavlja predviđanje modela za trenutačni primjer.
- Funkcija koštanja - izračunava se nakon što mreža generira predikciju i služi za mjerenje razlike između predviđenih vrijednosti i stvarnih oznaka. Vrsta funkcije koštanja bira se prema tipu problema; primjerice, za klasifikacijske zadatke često se koristi unakrsna entropija (engl. *cross entropy*), dok se kod regresijskih zadataka najčešće primjenjuje srednja kvadratna pogreška (engl. *mean square error*).
- Propagacija unatrag (engl. *backpropagation*) - proces tijekom kojeg se, na temelju izračunate funkcije koštanja, pogreška prenosi od izlaznog sloja mreže unatrag prema ulaznom sloju, a korištenjem lančanog pravila izračunavaju se gradijenti funkcije koštanja u odnosu na svaki parametar mreže, što omogućuje njihovo sustavno ažuriranje radi smanjenja ukupne pogreške modela. Gradijent u ovom kontekstu pokazuje u kojem smjeru i koliko je potrebno promijeniti svaki parametar kako bi se pogreška modela što učinkovitije smanjila.

- Optimizacija - parametri mreže (npr. težine i vrijednosti pomaka) ažuriraju se prema izračunatim gradijentima, najčešće korištenjem optimizacijskih algoritama poput gradijentnog spusta, Adam optimizatora ili RMSprop-a, s ciljem minimizacije funkcije koštanja i poboljšanja performansi modela.

Ovaj ciklus se ponavlja za svaki primjer iz skupa podataka ili za podskupinu primjera (engl. *batch*), tijekom više epoha, sve dok mreža ne postigne zadovoljavajuću razinu točnosti ili dok se ne ispuni neki od zadanih kriterija zaustavljanja.

#### 4.3.3. Regularizacija i hiperparametri

Duboke umjetne neuronske mreže, zbog velikog broja parametara, sklone su problemu pretreniranja, umanjujući sposobnost generalizacije na nove podatke. Kako bi se to spriječilo, koriste se različite tehnike regularizacije:

- L1 i L2 regularizacija - funkciji koštanja dodaje se dodatni član koji kažnjava velike vrijednosti parametara modela, čime se potiče jednostavnija i općenitija rješenja te smanjuje rizik od pretreniranja.
- Ispuštanje neurona (engl. *dropout*) - tijekom svakog prolaza kroz mrežu, nasumično se isključuje određeni postotak neurona čime se sprečava da model postane previše ovisan o pojedinim parametrima i potiče se učenje robusnijih i generaliziranih značajki.
- Rani prekid - treniranje se zaustavlja čim se performanse na validacijskom skupu počnu pogoršavati, što je znak početka pretreniranja.

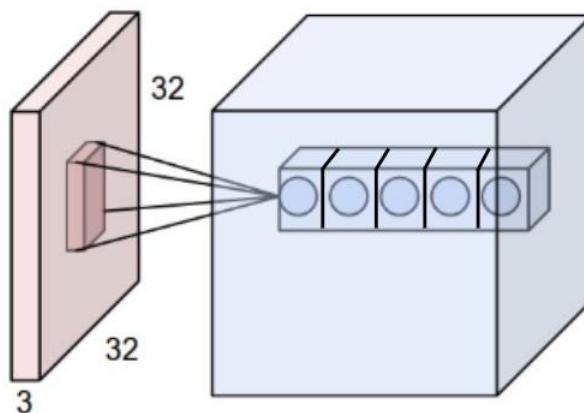
Osim regularizacije, uspješnost dubokih mreža uvelike ovisi o pravilnom odabiru hiperparametara, koji nisu izravno naučeni tijekom treniranja, već ih korisnik mora postaviti prije procesa učenja. Primjeri hiperparametara uključuju broj slojeva, broj neurona u svakom sloju, stopu učenja (engl. *learning rate*), veličinu podskupine primjera te broj epoha. Optimalni izbor ovih vrijednosti često se određuje eksperimentalno, koristeći metode poput ručnog ili nasumičnog pretraživanja.



#### 4.4. Konvolucijske neuronske mreže

Konvolucijske neuronske mreže čine posebnu podskupinu dubokih neuronskih mreža koje su optimizirane za obradu podataka rešetkaste strukture, kao što su slike, videozapisi ili drugi dvodimenzionalni i višedimenzionalni zapisi. Za razliku od klasičnih potpuno povezanih mreža, gdje je svaki neuron povezan sa svim neuronima prethodnog sloja, u CNN-ovima je svaki neuron povezan samo s ograničenim, lokalnim područjem ulaznih podataka. Takva lokalna povezanost omogućuje mreži da prepozna i izdvoji lokalne obrasce kao što su rubovi, tekture i jednostavni oblici, dok se istovremeno broj potrebnih parametara značajno smanjuje. Dodatno, svi neuroni u jednom sloju dijele iste težine, odnosno isti filter (engl. *kernel*), koji klizi po cijeloj slici i prepoznaje iste vrste značajki na različitim pozicijama, što omogućuje automatsko učenje ključnih vizualnih značajki bez potrebe za ručnim inženjeringom.

Važno je istaknuti da je svaki konvolucijski sloj trodimenzionalan. Kao što se vidi na slici 4.8., sastoji se od više filtara poslaganih jedan do drugoga (u ovom slučaju 5), pri čemu svaki filter ima svoj vlastiti skup težina. Horizontalno gledano, svaki neuron unutar sloja pokriva samo mali lokalni dio slike, odnosno malu regiju u svakoj od dubinskih komponenti (npr. u svim kanalima slike). Na taj način, svaki filter „skenira“ cijelu sliku, ali uvijek obrađuje lokalne informacije i na svim kanalima istovremeno, što omogućuje detekciju istog uzorka na različitim mjestima slike [25].

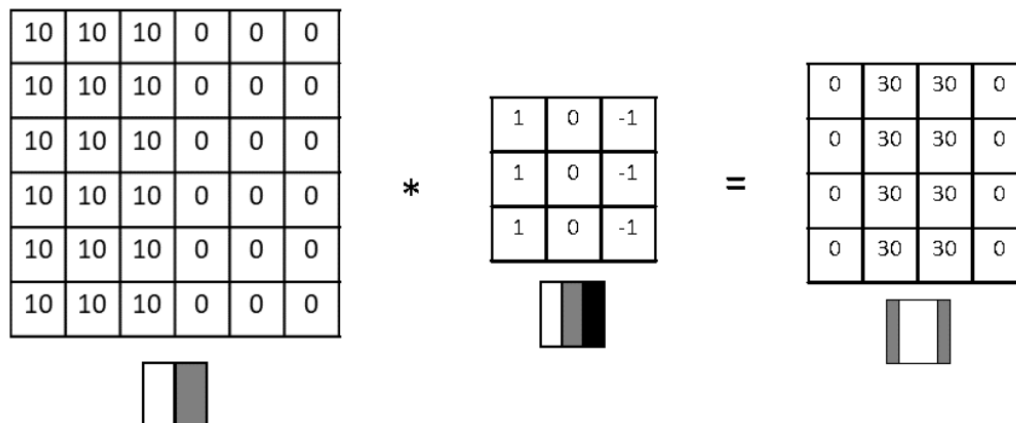


Slika 4.8. 3D prikaz konvolucijskog sloja [25]

#### 4.4.1. Slojevi i osnovne komponente CNN-a

Osnovna građevna jedinica CNN-a je konvolucijski sloj, u kojem se koristi skup filtara, odnosno malih matrica tipičnih dimenzija  $3 \times 3$  ili  $5 \times 5$ , koje se sustavno pomiču preko cijele slike. Svaki filter tijekom treniranja postaje specijaliziran za prepoznavanje određenih značajki, poput horizontalnih ili vertikalnih rubova, tekstura ili drugih vizualnih obrazaca, te za svaki svoj položaj na slici izračunava vrijednost koja odražava prisutnost te značajke. Rezultat tog procesa je mapa značajki (engl. *feature map*), koja prikazuje gdje se određeni uzorak pojavljuje unutar slike.

Primjerice, za detekciju vertikalnih rubova koristi se filter čije su vrijednosti postavljene tako da ističu nagle promjene intenziteta piksela u horizontalnom smjeru. Na slici 4.9. primijenjen je takav filter, pri čemu filter prolazi kroz svaki lokalni prozor slike i izračunava razliku između susjednih piksela u horizontalnom smjeru. Rezultat konvolucije je nova slika na kojoj su izraženi dijelovi gdje dolazi do nagle promjene svjetline, odnosno, tamo gdje se na izvornom prikazu nalaze vertikalni rubovi objekata [26].

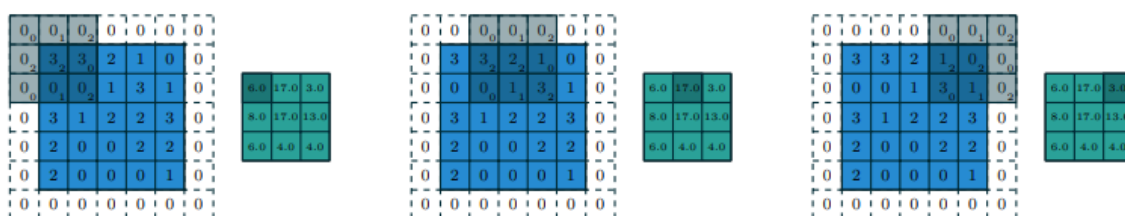


Slika 4.9. Primjena vertikalnog filtra [26]

Kako bi se tijekom konvolucije sačuvala izvorna veličina slike i omogućilo prepoznavanje značajki na samim rubovima, često se primjenjuje rubno popunjavanje (engl. *padding*). To je postupak dodavanja sloja nula oko rubova slike, čime se osigurava da i rubni pikseli budu obuhvaćeni filtrima te da veličina izlazne mape značajki ostane jednaka ulaznoj slici.

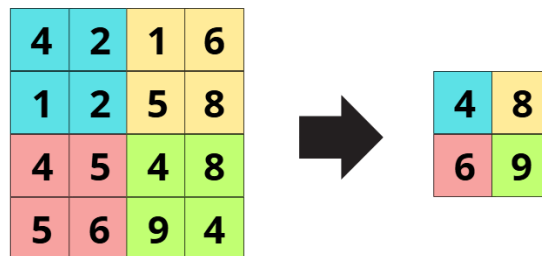
Istovremeno, parametar pomaka konvolucije (engl. *stride*) određuje za koliko se piksela filter pomiče u svakom koraku prelaska preko slike; veća vrijednost pomaka rezultira izlaznom mapom značajki manjih dimenzija i ubrzava smanjenje ukupne količine podataka kroz mrežu.

Na slici 4.10. prikazan je opisani proces konvolucije s rubnim popunjavanjem, odnosno dodavanjem sloja nula oko rubova slike te s korakom konvolucije pri kojem se filter pomiče za dva mjesta u svakom prolazu [27].



Slika 4.10. Prikaz rubnog popunjavanja i pomaka konvolucije [27]

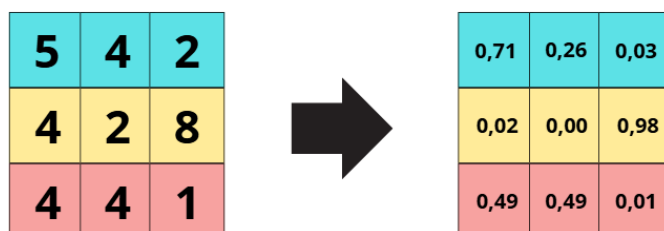
Nakon konvolucijskih slojeva obično slijedi *pooling* sloj, koji ima funkciju smanjenja prostornih dimenzija mapa značajki i sažimanja informacija iz manjih regija slike. Najčešće korišten je maksimalni *pooling* (engl. *max pooling*), gdje se iz svake podregije (npr.  $2 \times 2$ ) uzima maksimalna vrijednost, čime se ističu najizraženije značajke slike. Primjer ove vrste *poolinga* može se vidjeti na slici 4.11. Drugi česti pristup je prosječni *pooling* (engl. *average pooling*), kod kojeg se iz svake odabrane regije uzima prosječna vrijednost piksela. Maksimalni *pooling* je naročito koristan za isticanje dominantnih lokalnih značajki i povećava robusnost modela na translacije, šum i lokalne deformacije, dok prosječni *pooling* daje ujednačeniju, sažetu reprezentaciju regije. Obje metode doprinose učinkovitosti modela smanjenjem broja parametara i složenosti obrade.



Slika 4.11. Metoda maksimalnog poolinga

Nakon što konvolucijski i *pooling* slojevi izdvoje i sažmu relevantne značajke, rezultirajuće mape značajki se „ravnaju“ (engl. *flattening*) u jednodimenzionalni vektor. Taj vektor se zatim prosljeđuje kroz jedan ili više potpuno povezanih slojeva (engl. *fully connected layers*), gdje se svaka značajka kombinira s ostalima kako bi model mogao naučiti složenije veze i donijeti konačnu odluku ili predikciju. U ovim slojevima vrlo se često koristi ReLU aktivacijska funkcija jer uvodi nelinearnost, omogućuje bržu i stabilniju konvergenciju tijekom treniranja te učinkovito rješava problem nestajućeg gradijenta, što je posebno važno kod dubokih modela. ReLU također pomaže u očuvanju izraženijih značajki i smanjuje rizik od pretreniranja u složenim strukturama modela.

Na samom kraju CNN-a, posebno kod klasifikacijskih zadataka, koristi se *softmax* sloj. Softmax funkcija pretvara posljednji skup aktivacija modela u vjerojatnosti raspodijeljene po klasama, tako da njihov zbroj iznosi 1, kao što je prikazano na slici 4.12. To omogućuje interpretaciju izlaza mreže kao vjerojatnosti pripadnosti pojedinoj klasi, što je posebno korisno u višeklasnim problemima.



Slika 4.12. Primjena softmax funkcije

#### 4.4.2. Uloga CNN-a u prepoznavanju gesta ruke

Konvolucijske neuronske mreže iznimno su pogodne za sustave prepoznavanja gesta ruke jer mogu automatski izdvajati i učiti vizualne značajke iz slika, bez potrebe za ručnim inženjeringom značajki ili prethodnim znanjem o obliku ruke. Lokalna povezanost i dijeljenje težina čine ih otpornima na varijacije u položaju, veličini, orijentaciji i osvjetljenju ruke, dok višeslojna arhitektura omogućuje prepoznavanje i najsuptilnijih razlika u gestama. Osim toga, CNN-ovi su dovoljno učinkoviti za obradu podataka u stvarnom vremenu, što ih čini idealnim rješenjem za interaktivne sustave i primjene gdje su brzina i preciznost ključni. Zbog svih navedenih prednosti, CNN-ovi predstavljaju standard u modernim sustavima za prepoznavanje gesta temeljenim na računalnom vidu i središnju tehnologiju za razvoj prirodnih i intuitivnih sučelja između čovjeka i stroja.

Važno je naglasiti da se CNN-ovi ne koriste samo u završnoj fazi prepoznavanja gesta, već mogu biti primijenjeni u svim ključnim koracima sustava, uključujući detekciju, segmentaciju i praćenje ruke, čime dodatno doprinose cjelokupnoj učinkovitosti i pouzdanosti sustava.

## 5. Prototip sustava za gestualno upravljanje postavkama računala

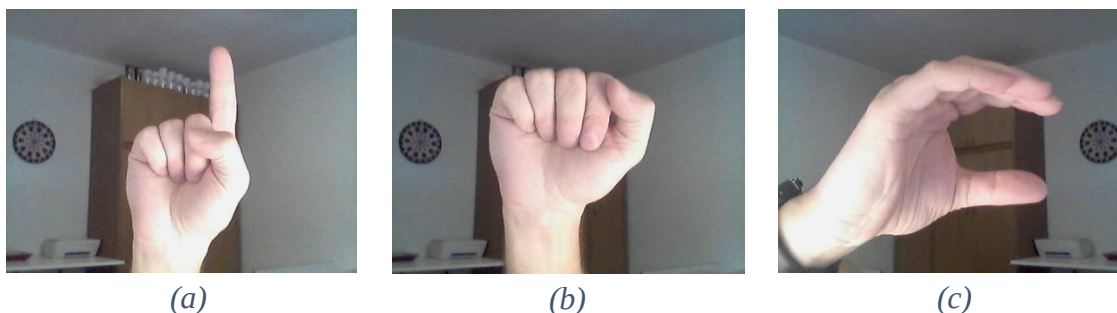
U ovom poglavlju detaljno je opisan razvoj funkcionalnog prototipa inovativnog sustava koji omogućuje upravljanje osnovnim postavkama računala, poput glasnoće zvuka i funkcija miša, isključivo gestama ruke, bez potrebe za dodatnim uređajima ili senzorima. U razvoju ovog sustava praktično su primijenjene metode obrađene u teoretskom dijelu rada, uključujući analizu i obradu slika, tehnike računalnog vida te metode dubokog učenja s posebnim naglaskom na konvolucijske neuronske mreže.

Za implementaciju sustava korišten je programski jezik Python, uz korištenje sljedećih ključnih biblioteka i alata:

- NumPy - za brzu i efikasnu manipulaciju numeričkim podacima i matricama, što je potrebno pri obradi slika i predikcija modela.
- TensorFlow i Keras - za izgradnju, treniranje i izvođenje dubokih konvolucijskih neuronskih mreža za klasifikaciju slika gesta ruke.
- ImageDataGenerator - za augmentaciju i pripremu skupova podataka tijekom treniranja modela.
- Matplotlib - za vizualizaciju rezultata treniranja, prikaz matrice zabune (engl. *confusion matrix*) i analizu performansi modela.
- Scikit-learn - za generiranje izvještaja o klasifikaciji i izradu matrice zabune tj. evaluacije uspješnosti modela.
- OpenCV - za dohvat i obradu slika s kamere u stvarnom vremenu, vizualizaciju rezultata i prikaz sučelja korisniku.
- MediaPipe - za pouzdano i brzo detektiranje i praćenje ruke u realnom vremenu koristeći geometrijske značajke šake.
- PyAutoGUI - za simulaciju pomaka pokazivača i klikanja mišem na razini operacijskog sustava.
- Pycaw - za upravljanje razinom sistemske glasnoće putem softverskog API-ja.
- Os i ctypes/comtypes - za upravljanje datotekama i integraciju s operacijskim sustavom.
- Vlastiti moduli (npr. `data_split`) - za pripremu i podjelu skupa podataka na treniranje, validaciju i testiranje.

Sustav je realiziran s podrškom za tri osnovne geste ruke:

- Gesta pokazivanja prstom (slika 5.1.a) - koristi se za upravljanje položajem pokazivača miša, omogućujući precizno pomicanje pokazivača na ekranu.
- Gesta šake tj. stisnuta šaka (slika 5.1.b) - služi za simulaciju klika mišem, što korisniku omogućuje interakciju s aplikacijama i elementima sučelja bez fizičkog miša.
- Gesta skaliranja (slika 5.1.c) - koristi se za kontrolu jačine zvuka, omogućujući jednostavno i brzo povećavanje ili smanjivanje glasnoće. U ovom radu skaliranjem se smatra približavanje, odnosno udaljšavanje palca i kažiprsta,



*Slika 5.1. Primjeri gesta: (a) pokazivanje prstom, (b) stisnuta šaka, (c) skaliranje*

Sustav je dizajniran s ciljem rada u stvarnom vremenu, otpornosti na promjene osvjetljenja i pozadine te osiguravanja jednostavnog i intuitivnog korisničkog iskustva. Ključna motivacija bila je omogućiti korisnicima brzu, pouzdanu i prirodnu interakciju s računalom koristeći isključivo pokrete ruku, čime se otvaraju nove mogućnosti za pristup i upravljanje digitalnim okruženjem na inovativan način.

## 5.1. Struktura podataka

Za potrebe treniranja i evaluacije modela prikupljeno je više od 25.000 slika ruku u raznim pozama i uvjetima. Dio slika snimljen je izravno kamerom računala na kojem će se sustav koristiti, čime je osigurana prilagodba modela stvarnim uvjetima korištenja. Kako bi se dodatno povećala raznolikost i otpornost modela na promjene osvjetljenja, pozadine i različite korisnike, dio podataka preuzet je iz javno dostupnog *Alicante Hand Gesture* dataseta [28]. Time se poboljšava robusnost sustava i smanjuje mogućnost pogrešnog prepoznavanja u realnim situacijama.

Prikupljeni podaci dijele se na stvarne (snimljene kamerom) i sintetički generirane slike. Nakon inicijalnog pregleda i sortiranja, sve slike grupirane su u pet kategorija (engl. *foldera*), prema gesti koja je prikazana ili odsutnosti ruke:

- *Fist* – slike stisnute šake (za klik mišem)
- *Scale* – slike geste skaliranja (za upravljanje glasnoćom)
- *Point* – slike pokazivanja prstom (za pomicanje pokazivača)
- *Other* – slike drugih, nerelevantnih gesta ruke
- *None* – slike u kojima nema prikaza ruke

Prva tri foldera sadrže primjere reprezentativnih gesta koje se izravno mapiraju na funkcije sustava, dok su kategorije *Other* i *None* uvedene kako bi model bio sposoban razlikovati relevantne geste od ostalih pokreta ruke ili situacija kada ruka uopće nije prisutna u kadru. To dodatno smanjuje broj lažno pozitivnih prepoznavanja i doprinosi pouzdanosti sustava.

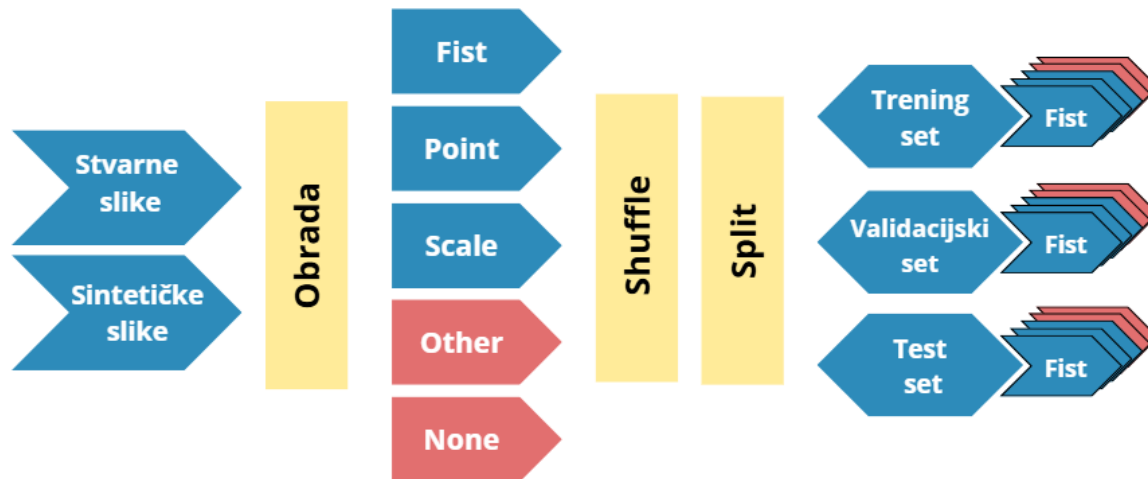
Slike iz svih kategorija dodatno se obrađuju i skaliraju na jedinstvenu dimenziju, čime se osigurava standardizirani ulaz u neuronsku mrežu i olakšava proces učenja modela. Nakon obrade, podaci se automatski dijele na tri podskupa: trening skup, koji se koristi za učenje modela, validacijski skup, namijenjen praćenju i optimizaciji performansi tijekom treniranja, te testni skup, koji služi za konačnu evaluaciju uspješnosti modela na neviđenim podacima. Omjeri podjele, primjerice 70 % za trening, 15 % za validaciju i 15 % za test, definirani su u programskom kodu, a sama podjela podataka provodi se samo ako direktorij za podjelu podataka (*datasplit*) ne postoji, odnosno prilikom prvog pokretanja ili nakon ručnog brisanja tog direktorija. Time se značajno štedi vrijeme pri ponovljenim pokretanjima i testiranjima sustava. Prije same podjele, podaci unutar svake klase



se nasumično miješaju pomoću `random.shuffle()`, čime se osigurava ravnomjerna i nepristrana raspodjela slika u svaki od setova.

```
def split_dataset(base_dir, output_base, train_ratio, val_ratio, test_ratio):
    if os.path.exists(output_base):
        print(f"Split dataset folder '{output_base}' already exists, skipping split.")
        return
    classes = [d for d in os.listdir(base_dir) if
os.path.isdir(os.path.join(base_dir, d))]
    for split in ['train', 'val', 'test']:
        for class_name in classes:
            os.makedirs(os.path.join(output_base, split, class_name),
exist_ok=True)
        for class_name in classes:
            src_folder = os.path.join(base_dir, class_name)
            images = [f for f in os.listdir(src_folder) if
os.path.isfile(os.path.join(src_folder, f))]
            random.shuffle(images)
            n = len(images)
            n_train = int(train_ratio * n)
            n_val = int(val_ratio * n)
            n_test = n - n_train - n_val
            train_files = images[:n_train]
            val_files = images[n_train:n_train+n_val]
            test_files = images[n_train+n_val:]
            for file_list, split in [(train_files, 'train'), (val_files, 'val'),
(test_files, 'test')]:
                for file_name in file_list:
                    src = os.path.join(src_folder, file_name)
                    dst = os.path.join(output_base, split, class_name, file_name)
                    shutil.copy2(src, dst)
        print("Dataset split completed!")
```

Konačna struktura podataka sastoji se od tri glavna direktorija: train, val i test, a svaki od njih organiziran je u istih pet kategorija originalnog seta podataka: *Fist*, *Scale*, *Point*, *Other*, *None*. To omogućuje precizno i sustavno upravljanje podacima te olakšava provedbu svih faza treniranja, validacije i testiranja modela. Cijela ova struktura, od obrade podataka u kategorije, do podjele na setove, prikazana je na slici 5.2.



Slika 5.2. Struktura podataka

Ovakav način organizacije podataka osigurava transparentnost, ponovljivost eksperimenata i visoku učinkovitost procesa učenja modela.

## 5.2. CNN model

Nakon što se podaci iz svih kategorija nasumično izmiješaju i raspodijele u trening, validacijski i testni skup prema zadanim omjerima, slijedi postupak augmentacije podataka, koji je ključan za kvalitetno treniranje CNN-a. Augmentacija se primjenjuje isključivo na trenirajućem skupu slika, s ciljem povećanja raznolikosti i količine podataka bez potrebe za dodatnim ručnim prikupljanjem novih uzoraka. Na taj se način model čini otpornijim na različite varijacije koje se mogu pojaviti u stvarnim uvjetima korištenja.

Augmentacija uključuje niz transformacija, među kojima su:

- normalizacija piksela - sve slike se skaliraju tako da vrijednosti piksela budu u rasponu [0, 1], koristeći parametar *rescale*. Time se standardizira ulaz u neuronsku mrežu i olakšava proces konvergencije tijekom učenja.
- nasumična rotacija - parametar *rotation\_range* omogućuje nasumično rotiranje slika do  $\pm 20$  stupnjeva. Ova transformacija simulira različite kuteve gledanja ruke i čini model otpornijim na varijacije u orijentaciji geste.

- pomak po širini i visini - parametri *width\_shift\_range* i *height\_shift\_range* omogućuju pomicanje slike po vodoravnoj i okomitoj osi do 10 % ukupne širine ili visine slike. Time model uči prepoznavati geste bez obzira na to gdje se ruka nalazi unutar kadra.
- širenje - parametar *shear\_range* primjenjuje nasumičnu paralelogramsku deformaciju slike do 15 %. Ova transformacija pomaže modelu da postane otporniji na manje geometrijske distorzije koje nastaju zbog perspektive ili pokreta ruke.
- zumiranje - parametar *zoom\_range* omogućuje nasumično povećavanje ili smanjivanje slike do 15 %. Time model postaje otporniji na različite udaljenosti ruke od kamere i promjene u veličini prikazane geste.
- promjena svjetline - parametar *brightness\_range* nasumično mijenja svjetlinu slike u zadanom rasponu. Ova augmentacija simulira različite uvjete osvjetljenja i čini model robusnijim na promjene svjetla.
- horizontalno zrcaljenje - parametar *horizontal\_flip* omogućuje nasumično horizontalno zrcaljenje slike, što pomaže modelu u učenju prepoznavanja gesti lijeve i desne ruke.

Ove transformacije značajno povećavaju varijabilnost skupa podataka i omogućuju modelu bolje generalizacijske sposobnosti na neviđene primjere iz stvarnog svijeta.

```
# Data augmentation for training
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    shear_range=0.15,
    zoom_range=0.15,
    brightness_range=(0.7, 1.3),
    horizontal_flip=True,
    fill_mode='nearest',
    preprocessing_function=lambda x: tf.image.random_crop(x,
size=(64,64,3)) # random cropping
)
```

Važno je naglasiti da se validacijski i testni skupovi ne podvrgavaju nikakvoj augmentaciji, osim osnovne normalizacije piksela, čime se osigurava objektivna procjena stvarnih performansi modela i njegove sposobnosti generalizacije.

Model za prepoznavanje gesti temelji se na konvolucijskoj neuronskoj mreži izgrađenoj kao slijedni (engl. *sequential*) model. Na ulazu model prima slike dimenzija 64x64 piksela s tri kanala (RGB), što je standardna veličina prilagođena prikupljenom skupu podataka.

Arhitektura modela sastoji se od četiri glavna konvolucijska bloka. Svaki blok započinje konvolucijskim slojem koji ima sve više filtera kako idemo dublje u mrežu. Prvi sloj ima 32 filtera, drugi 64, treći 128, a četvrti 256 filtera. Ovi konvolucijski slojevi služe za detekciju sve složenijih značajki na slici, od osnovnih rubova i tekstura do apstraktnih uzoraka karakterističnih za određene geste. Nakon svakog konvolucijskog sloja nalazi se sloj za normalizaciju podskupine primjera (*BatchNormalization*), koji stabilizira i ubrzava proces treniranja, te sloj za maksimalni *pooling* (*MaxPooling*), kojim se smanjuje dimenzija slike i tako smanjuje količina podataka koju je potrebno obraditi u sljedećim slojevima.

Četvrti konvolucijski sloj dodatno koristi L2 regularizaciju kako bi se smanjila mogućnost pretreniranja modela. Nakon posljednjeg bloka nalazi se *GlobalAveragePooling2D* sloj, koji iz svake mape značajki izračunava prosječnu vrijednost, čime se dodatno smanjuje broj parametara i čuva prostorna neovisnost modela.

Nakon toga slijedi *Dropout* sloj koji nasumično isključuje 40 % neurona kako bi se spriječila pretreniranost, a zatim i gusto povezani *Dense* sloj s 128 neurona i ReLU aktivacijom. Ovaj sloj kombinira sve naučene značajke iz prethodnih slojeva i priprema ih za završnu klasifikaciju. Dodatni *Dropout* sloj (30 %) dodatno povećava otpornost modela na pretreniranost.

Na kraju se nalazi izlazni sloj koji ima onoliko neurona koliko ima klasa gesti (u ovom modelu 5), a koristi softmax aktivacijsku funkciju kako bi model mogao vratiti vjerojatnosti za svaku od klasa. Ovaj sloj također koristi L2 regularizaciju radi veće robusnosti.

Model se trenira korištenjem optimizatora Adam, koji omogućuje brzo i efikasno učenje, te kategoričkom unakrsnom entropijom kao funkcijom gubitka, koja je prilagođena za zadatke višeklasne klasifikacije poput ovog.

Ovakva arhitektura omogućuje modelu da učinkovito uči karakteristike svake geste i pouzdano ih prepoznaje na novim, neviđenim slikama.

```
# Define CNN model
model = Sequential([
    Input(shape=(64, 64, 3)),
    Conv2D(32, (3, 3), padding='same', activation='relu'),
    BatchNormalization(),
    MaxPooling2D(),

    Conv2D(64, (3, 3), padding='same', activation='relu'),
    BatchNormalization(),
    MaxPooling2D(),

    Conv2D(128, (3, 3), padding='same', activation='relu'),
    BatchNormalization(),
    MaxPooling2D(),

    Conv2D(256, (3, 3), padding='same', activation='relu',
kernel_regularizer=l2(5e-5)),
    BatchNormalization(),

    GlobalAveragePooling2D(),
    Dropout(0.4),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax', kernel_regularizer=l2(5e-5))
])

model.compile(
    loss='categorical_crossentropy',
    optimizer=tf.keras.optimizers.Adam(learning_rate=3e-4),
    metrics=['accuracy']
)
```

Za poboljšanje procesa treniranja koriste se tri ključna povratna poziva (engl. *callback*):

- *ReduceLROnPlateau* - automatski smanjuje stopu učenja kada se validacijska točnost ne poboljšava određeni broj epoha, što pomaže modelu da se efikasnije prilagodi minimumu funkcije gubitka.
- *EarlyStopping* - prati validacijsku točnost i zaustavlja treniranje ako se ona ne poboljša unutar zadanog broja epoha. Također vraća najbolje naučene težine, čime se sprečava pretreniranost i nepotrebno treniranje.

- *ModelCheckpoint* - sprema težine modela svaki put kad se postigne nova najbolja validacijska točnost. Time se osigurava da završni model uvijek predstavlja najbolje naučene parametre čak i ako dođe do pada performansi u kasnijim epohama.

Ove callback funkcije zajedno omogućuju stabilnije, brže i učinkovitije treniranje modela, uz automatsko sprečavanje pretreniranosti i optimalan odabir najboljeg modela.

```
# --- Callbacks: always evaluate the best epoch ---
ckpt_path = 'best_gesture_model.keras'
callbacks = [
    ReduceLROnPlateau(monitor='val_accuracy', factor=0.5, patience=3,
min_lr=1e-5, verbose=1),
    EarlyStopping(monitor='val_accuracy', patience=6,
restore_best_weights=True, verbose=1),
    ModelCheckpoint(ckpt_path, monitor='val_accuracy', save_best_only=True,
verbose=1)
]
```

### 5.3. Arhitektura sustava i implementacija

Sustav za upravljanje postavkama računala gestama ruke temelji se na integraciji nekoliko ključnih komponenti računalnog vida, strojnog učenja i kontrole operacijskog sustava. Osnovni tijek rada sustava može se opisati kroz nekoliko glavnih faza:

1. Dohvat slike i detekcija ruke - sustav koristi kameru za kontinuirano dohvaćanje video okvira koristeći *cv2.VideoCapture()*. Svaki okvir se obrađuje korištenjem *MediaPipe* biblioteke, točnije funkcije *hands.process()*, koja detektira položaj i orijentaciju ruke te precizno locira ključne točke (engl. *landmarks*), pri čemu se koriste ručno postavljeni parametri poput broja očekivanih ruku i razine složenosti modela.

```
# Camera
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("✗ Error: Cannot open camera"); return
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
```

```
# ----- MediaPipe (downscaled) -----
    rgb_small = cv2.cvtColor(cv2.resize(frame_bgr, MP_SMALL),
cv2.COLOR_BGR2RGB)
    results = hands.process(rgb_small)
    have_hand = results.multi_hand_landmarks is not None
    lm_norm = results.multi_hand_landmarks[0].landmark if have_hand else
None # take Landmarks of the first hand
```

2. Izrezivanje i priprema slike - na temelju precizno detektiranih ključnih točaka na ruci određuje se područje slike (*bbox\_now*) koje obuhvaća cijelu šaku. Ovo područje se zatim izdvaja iz originalnog kadra, proporcionalno proširuje kako bi obuhvatilo cijelu ruku bez gubitka informacija, i skalira na standardiziranu dimenziju od 64x64 piksela, sve unutar funkcije *crop\_letterbox\_bgr()*. Nakon skaliranja, slika se po potrebi dodatno centrirano smješta unutar platna odgovarajuće veličine, dok se preostali prostor popunjava crnom bojom, čime se zadržava prostorna konzistentnost svih ulaza u model. Konačno, slika se pretvara u RGB format i normalizira na vrijednosti piksela između 0 i 1 koristeći *preprocess\_for\_cnn\_from\_bgr()* funkciju

```
# ----- Helpers -----
def bbox_from_landmarks(lm_norm, pad=0.35):
    xs = [lm.x for lm in lm_norm]; ys = [lm.y for lm in lm_norm]
    if not xs or not ys: return None
    x1, y1 = max(0.0, min(xs) - pad), max(0.0, min(ys) - pad)
    x2, y2 = min(1.0, max(xs) + pad), min(1.0, max(ys) + pad)
    w, h = x2 - x1, y2 - y1
    if w <= 0 or h <= 0: return None
    # make square
    if w > h:
        d = (w - h) / 2; y1 = max(0.0, y1 - d); y2 = min(1.0, y2 + d)
    else:
        d = (h - w) / 2; x1 = max(0.0, x1 - d); x2 = min(1.0, x2 + d)
    return [x1, y1, x2, y2]

def crop_letterbox_bgr(frame_bgr, bbox_norm, out_size=(64, 64)):
    H, W = frame_bgr.shape[:2]
    x1, y1, x2, y2 = bbox_norm
    X1, Y1 = int(x1*W), int(y1*H)
    X2, Y2 = int(x2*W), int(y2*H)
    X1 = max(0, min(W-1, X1)); X2 = max(0, min(W, X2))
    Y1 = max(0, min(H-1, Y1)); Y2 = max(0, min(H, Y2))
```

```

if X2 <= X1 or Y2 <= Y1: return None
roi = frame_bgr[Y1:Y2, X1:X2]
h, w = roi.shape[:2]; th, tw = out_size
scale = min(tw/w, th/h)
nw, nh = int(w*scale), int(h*scale)
resized = cv2.resize(roi, (nw, nh))
canvas = np.zeros((th, tw, 3), dtype=np.uint8)
x_off, y_off = (tw-nw)//2, (th-nh)//2
canvas[y_off:y_off+nh, x_off:x_off+nw] = resized
return canvas

def preprocess_for_cnn_from_bgr(bgr_img, img_size=None):
    if bgr_img is None or bgr_img.size == 0:
        return None
    rgb = cv2.cvtColor(bgr_img, cv2.COLOR_BGR2RGB)

    # Resize only if not already the right size
    if img_size is not None:
        h, w = rgb.shape[:2]
        if (h, w) != img_size:
            rgb = cv2.resize(rgb, (img_size[1], img_size[0])) # (W, H)

    x = rgb.astype(np.float32) / 255.0 # <-- normalization [0,1]
    return x[None, ...]

```

3. Klasifikacija geste - izrezana i obrađena slika ruke predaje se unaprijed istreniranom modelu CNN-a, koji klasificira gestu u jednu od definiranih klasa (*point*, *fist*, *scale*, *other*, *none*). Rezultat klasifikacije koristi se za daljnje upravljanje.

```

p = model.predict(x, verbose=0)[0] # raw probs
pred_idx = int(np.argmax(p))
pred_lbl = idx_to_class.get(pred_idx, 'None')
conf = float(p[pred_idx])

```

4. Interpretacija i mapiranje gesta na radnje - pokazivanje prstom (*Point*) koristi se za upravljanje pozicijom pokazivača miša, gdje se pokret dlana projicira na ekran. Stisnuta šaka (*Fist*) prepoznaje se kao naredba za klik mišem samo ako joj prethodi gesta pokazivanja prstom. Skaliranje (*Scale*) omogućuje upravljanje glasnoćom zvuka, koristeći udaljenost između palca i kažiprsta za precizno podešavanje razine glasnoće.



5. Izvršavanje radnji na sustavu - ovisno o prepoznatoj gesti, sustav koristi PyAutoGUI biblioteku za simulaciju pokreta miša i klikanja, dok se za kontrolu glasnoće na Windowsu koristi pycaw modul koji omogućuje izravno podešavanje sistemske glasnoće.

```
if active_gesture == 'Point' and have_hand:
    # Compute the palm center as the average of some Landmark
    coordinates
    cx = np.mean([lm_norm[i].x for i in PALM_IDXS])
    cy = np.mean([lm_norm[i].y for i in PALM_IDXS])
    # Convert normalized hand coordinates into pixel coordinates of
    the screen.
    x_raw = int(cx * screen_w)
    y_raw = int(cy * screen_h)
    # Apply exponential smoothing with factor alpha_cursor
    x_s = int((1 - alpha_cursor) * x_s + alpha_cursor * x_raw)
    y_s = int((1 - alpha_cursor) * y_s + alpha_cursor * y_raw)
    # Only move the mouse if the new smoothed position differs enough
    from the previous position.
    if abs(x_s - prev_x) + abs(y_s - prev_y) > deadzone:
        pyautogui.moveTo(x_s, y_s, duration=0)
        prev_x, prev_y = x_s, y_s
    # ----- Single-click edge trigger while pointing -----
    if active_gesture == 'Point':
        if not fist_high and p[IDX_FIST] >= FIST_PRESS_T_POINT and
click_cooldown == 0:
            # rising edge => click once
            pyautogui.click()
            click_cooldown = CLICK_COOLDOWN
            cv2.putText(frame, 'CLICK!', (10, 86),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 3)
            fist_high = True
        elif fist_high and p[IDX_FIST] <= FIST_RELEASE_T_POINT:
            # re-arm after dropping below release threshold
            fist_high = False
    else:
        # if not pointing, just re-arm based on release
        if p[IDX_FIST] <= FIST_RELEASE_T_POINT:
            fist_high = False
elif active_gesture == 'Scale' and have_hand and VOLUME_CONTROL_AVAILABLE and
volume is not None:
    H, W = frame.shape[:2]
    thumb_tip = lm_norm[mp_hands.HandLandmark.THUMB_TIP]
```

```

index_tip = lm_norm[mp_hands.HandLandmark.INDEX_FINGER_TIP]
index_mcp = lm_norm[mp_hands.HandLandmark.INDEX_FINGER_MCP]
pinky_mcp = lm_norm[mp_hands.HandLandmark.PINKY_MCP]
# Calculates the Euclidean distance (in pixels) between thumb tip
and index tip
pinch_px = np.hypot((thumb_tip.x - index_tip.x) * W, (thumb_tip.y
- index_tip.y) * H)
# Measures the width of your hand in pixels
hand_w_px = max(1.0, np.hypot((index_mcp.x - pinky_mcp.x) * W,
(index_mcp.y - pinky_mcp.y) * H))
norm = pinch_px / hand_w_px

# dynamic calibration
# creates a starting range based on your hand's first position.
if calib_min is None or calib_max is None:
    calib_min, calib_max = norm * 0.6, norm * 1.4
else:
    calib_min = min(calib_min * 0.995 + norm * 0.005, norm)
    calib_max = max(calib_max * 0.995 + norm * 0.005, norm)
    if calib_max - calib_min < 0.10:
        calib_min -= 0.05; calib_max += 0.05

# shrink mapping range so ends are easier
# find the current calibration window center & half-width
mid = 0.5 * (calib_min + calib_max)
half = 0.5 * (calib_max - calib_min) * RANGE_SHRINK
map_min, map_max = mid - half, mid + half
norm_c = float(np.clip(norm, map_min, map_max))
target = np.interp(norm_c, [map_min, map_max], [0, 100])

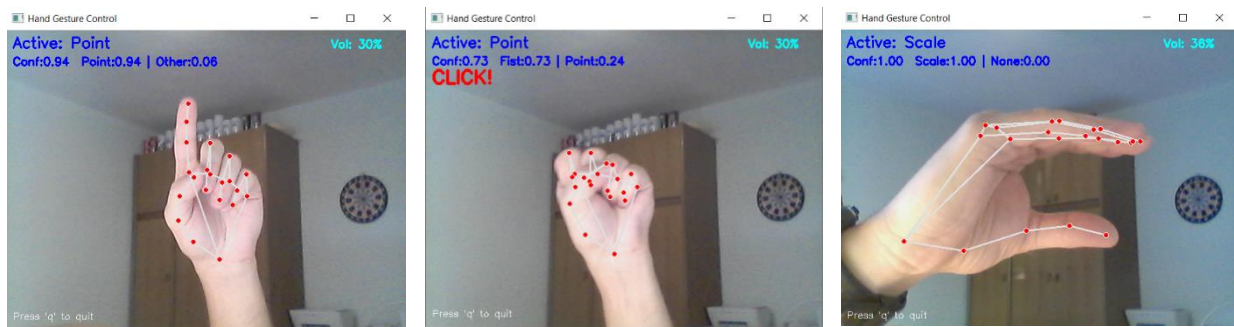
# snap near edges + smoothing
if target >= 100 - EDGE_SNAP: target = 100.0
elif target <= EDGE_SNAP: target = 0.0

if entered_scale: smoothed_vol = target
else:
    alpha = ALPHA_VOL if (EDGE_SNAP < target < 100-EDGE_SNAP)
else min(0.75, ALPHA_VOL + 0.35)
    smoothed_vol = (1 - alpha) * smoothed_vol + alpha * target

#vol_to_set = int(np.ceil(smoothed_vol))
vol_to_set = int(round(smoothed_vol / 2.0) * 2)
set_volume_from_scalar(volume, vol_to_set)

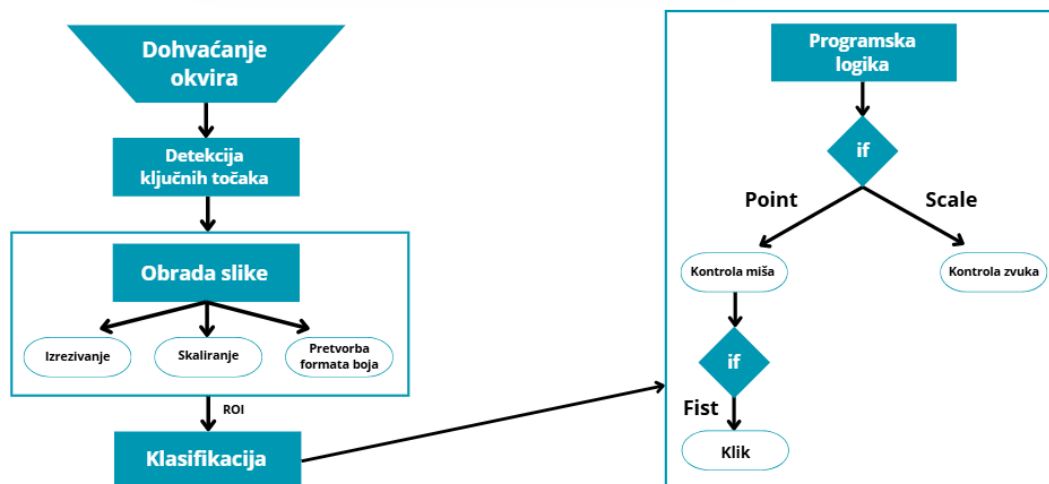
```

6. Korisničko sučelje i povratna informacija - korisniku se u stvarnom vremenu prikazuje video s detektiranom rukom, prepoznatom gestom i trenutnim statusom sustava, što omogućuje praćenje i jednostavnu interakciju. Na slici 5.3. prikazano je korisničko sučelje za sve tri geste koje sustav prepoznaje s vrlo jasnim statusom sustava.



Slika 5.3. Korisničko sučelje sustava za kontrolu postavki gestama ruke

Sve ove komponente zajedno omogućuju rad sustava u stvarnom vremenu, bez potrebe za dodatnom opremom osim kamere, te pružaju intuitivno i prirodno upravljanje računalom gestama ruke. Pojednostavljena schema arhitekture prikazana je na slici 5.4.



Slika 5.4. Arhitektura sustava za kontrolu postavki gestama ruke

#### 5.4. Dinamičko upravljanje postavkama

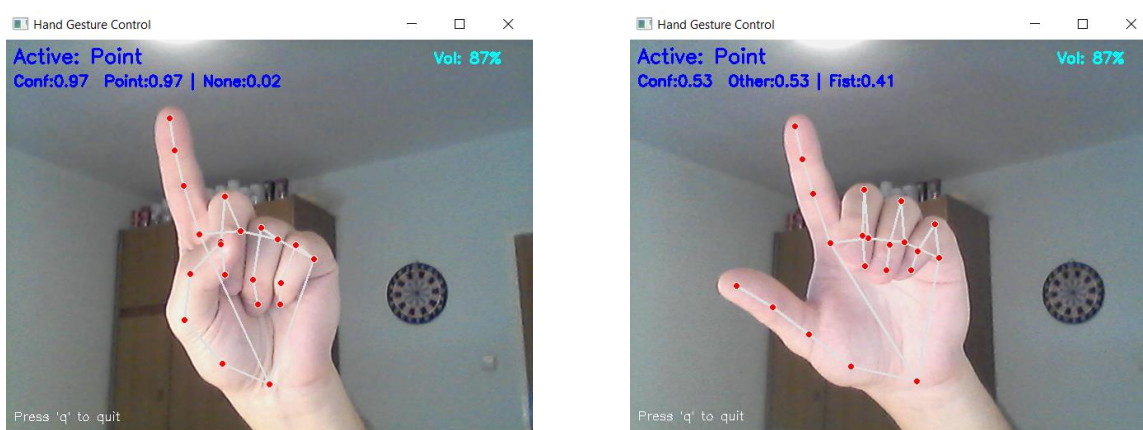
Upravljanje gestama u realnom vremenu zahtijeva ne samo točnu detekciju, već i sofisticiran način prijelaza između različitih gesti kako bi korisničko iskustvo bilo pouzdano i prirodno. Ključna komponenta sustava je logika koja određuje kada se jedna gesta smatra aktivnom, a kada se prelazi u drugo stanje ili se deaktivira. U tu svrhu koristi se princip histereze, odnosno dvostrukih pragova: za ulazak u stanje i za izlazak iz njega.

Kada korisnik izvede, primjerice, gestu pokazivanja (*Point*), model ne reagira trenutačno na svaku malu promjenu u vjerojatnosti, već zahtijeva da ta vjerojatnost kroz nekoliko uzastopnih okvira ostane iznad određenog ulaznog praga. Tek tada gesta postaje aktivna i omogućuje korisniku upravljanje pokazivačem miša. Slično, kada korisnik pomakne ruku ili promijeni gestu, sustav ne prekida odmah aktivnu gestu nego čeka da vjerojatnost padne ispod nižeg izlaznog praga, također kroz nekoliko uzastopnih frameova. Ovakav način prijelaza, temeljen na *dwell*, *enter* i *exit* brojačima, smanjuje slučajne promjene stanja uzrokovane šumom, lošom detekcijom ili brzim prolaznim pokretima ruke. Pragovi korišteni u sustavu za svaku od gesta navedeni su u tablici 5.1.

Tablica 5.1. Pragovi za promjenu stanja sustava

| Stanje | Ulazni prag<br>(ENTER_T) | Izlazni prag<br>(EXIT_T) | Dwell (ulaz) | Dwell (izlaz) | Posebni<br>uvjeti                     |
|--------|--------------------------|--------------------------|--------------|---------------|---------------------------------------|
| Point  | 0.6                      | 0.3                      | 2 okvira     | 2 okvira      | Minimalno<br>zadržavanje:<br>5 okvira |
| Scale  | 0.6                      | 0.35                     | 2 okvira     | 2 okvira      |                                       |
| Fist   | 0.5                      | 0.2                      | -            | -             | Detekcija<br>ruba za klik             |

Ovakva razlika između ulaznog i izlaznog praga, poznata kao histereza, osigurava da manje oscilacije u klasifikacijskim rezultatima ne uzrokuju neželjene prekide ili neočekivane promjene u prepoznatoj gesti. Dodatno, minimalno trajanje pojedinih stanja, poput *Point* stanja, doprinosi stabilnosti upravljanja, omogućujući korisniku da zadrži kontrolu nad pokazivačem čak i ako dođe do kratkotrajnog gubitka detekcije. Na slici 5.5. prikazana je otpornost sustava na slučajne pokrete poput podizanja palca. Iako je vjerojatnost da je detektirana nova gesta poprilično velika, minimalno zadržavanje stanja od 5 okvira onemogućuje promjenu stanja.



Slika 5.5. Otpornost sustava na slučajne pokrete

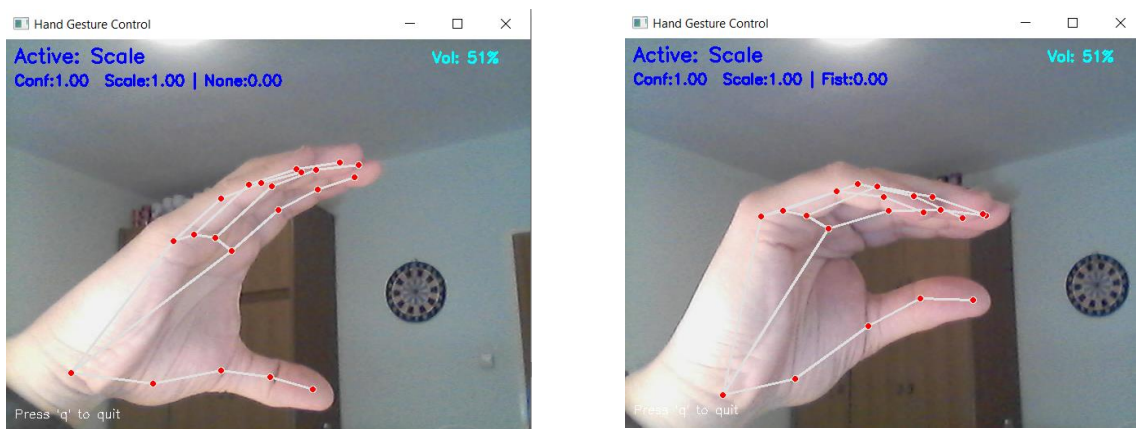
Gesta *Scale*, namijenjena upravljanju glasnoćom, koristi slične pragove, s dodatnim mehanizmom za automatsku i dinamičku kalibraciju: sustav na temelju prvih nekoliko trenutaka u kojima je gesta aktivna određuje raspon između najbližeg i najudaljenijeg položaja prstiju, te taj raspon tijekom korištenja kontinuirano i prilagodljivo osvježava kroz varijable *calib\_min* i *calib\_max*. To omogućuje svakom korisniku, bez obzira na veličinu ruke ili udaljenost od kamere, jednaku preciznost i intuitivnost u upravljanju glasnoćom.

```
# dynamic calibration
# creates a starting range based on your hand's first position.
if calib_min is None or calib_max is None:
    calib_min, calib_max = norm * 0.6, norm * 1.4
else:
    calib_min = min(calib_min * 0.98 + norm * 0.02, norm)
    calib_max = max(calib_max * 0.98 + norm * 0.02, norm)
    if calib_max - calib_min < 0.10:
```

```
calib_min -= 0.05; calib_max += 0.05
```

```
# shrink mapping range so ends are easier  
# find the current calibration window center & half-width  
mid = 0.5 * (calib_min + calib_max)  
half = 0.5 * (calib_max - calib_min) * RANGE_SHRINK  
map_min, map_max = mid - half, mid + half  
norm_c = float(np.clip(norm, map_min, map_max))  
target = np.interp(norm_c, [map_min, map_max], [0, 100])
```

Na slici 5.6. je prikazan primjer dinamičke kalibracije raspona ruke, gdje program automatski prepoznaje i ažurira minimalni i maksimalni razmak između prstiju prema stvarnim kretnjama korisnika. Zahvaljujući ovom pristupu, moguće je postići istu razinu zvuka čak i kada različiti korisnici, ili isti korisnik u različitim uvjetima, imaju vidljivo različite raspone otvaranja ruke.



Slika 5.6. Dinamička kalibracija raspona ruke

Klik mišem izveden stisnutom šakom (*Fist*) dok je aktivna gesta pokazivanja (*Point*) temelji se na detekciji ruba, odnosno na korištenju dva praga vjerojatnosti. Klik se izvršava samo u trenutku kad vjerojatnost za gestu *Fist* prvi put prijeđe viši prag (npr. 0.5), čime se detektira prijelaz iz otvorene u stisnutu šaku. Nakon toga, novi klik nije moguć dok vjerojatnost ne padne ispod nižeg praga (npr. 0.2). Ovo sprečava višestruko okidanje na jedno stiskanje šake, dok dodatna varijabla *cooldown* uvodi kratku vremensku blokadu između klikova radi dodatne zaštite od neželjenih višestrukih signala.

```
# cooldown
if click_cooldown > 0: click_cooldown -= 1
```

Stabilnost vizualnog i funkcionalnog odgovora dodatno je postignuta eksponencijalnim izravnavanjem, najviše izraženim kod upravljanja pokazivačem. Umjesto da se pokazivač miče točno u skladu sa svakim novim položajem ruke, primjenjuje se tzv. *alpha\_cursor* parametar, kojim se određuje koliko brzo i koliko glatko sustav reagira na promjene. Veća vrijednost alpha parametra vodi bržem, ali potencijalno drhtavijem kretanju pokazivača, dok niža vrijednost daje glađu, ali nešto sporiju reakciju. Na taj način, korisnik ima osjećaj kontrole i preciznosti, čak i kada detekcija pokreta ruke nije savršena.

Za dodatnu otpornost na šum, uvedena je „mrtva zona” (engl. *deadzone*), zahvaljujući kojoj se kursor na ekranu neće pomaknuti za svaki mikro pokret ili vibraciju ruke, nego samo kada je pomak dovoljno izražen. Slični principi koriste se i u detekciji ostalih komandi, čime se jamči da će korisnik doživjeti očekivano ponašanje svakog puta kad izvede određenu gestu.

```
# Apply exponential smoothing with factor alpha_cursor
x_s = int((1 - alpha_cursor) * x_s + alpha_cursor * x_raw)
y_s = int((1 - alpha_cursor) * y_s + alpha_cursor * y_raw)
# Only move the mouse if the new smoothed position differs enough
from the previous position.
if abs(x_s - prev_x) + abs(y_s - prev_y) > deadzone:
    pyautogui.moveTo(x_s, y_s, duration=0)
    prev_x, prev_y = x_s, y_s
```

Svi ovi mehanizmi zajedno omogućuju sustavu da dinamički i automatski prilagođava svoje interne postavke, osiguravajući optimalno i pouzdano upravljanje bez potrebe za dodatnim rukovanjem ili konfiguracijom od strane korisnika. Istovremeno, vizualna povratna informacija o trenutačno aktivnoj gesti, sigurnosti detekcije i vrijednosti parametara (poput glasnoće) čini rad transparentnim i intuitivnim. Svaku od gesti moguće je ilustrirati stvarnim slikama iz rada sustava; na primjer, za *Point* je jasno vidljiv centar dlana i smjer pokreta kursora, za *Scale* se ističe mjerenje udaljenosti između prstiju i promjena glasnoće, dok *Fist* gesta okida klik što je vizualno označeno na ekranu. Sve ove prilagodbe i zaštite čine sustav robusnim, praktičnim i primjenjivim u svakodnevnim scenarijima.

## 6. Evaluacija sustava za kontrolu postavki gestama ruke

### 6.1. Mjere točnosti

U procesu evaluacije sustava za prepoznavanje gesti ključno je koristiti prikladne metrike koje omogućuju objektivnu procjenu točnosti i učinkovitosti modela. U ovom su sustavu korištene četiri osnovne mjere evaluacije: preciznost (engl. *precision*), odziv (engl. *recall*), F1-mjera i točnost (engl. *accuracy*). Svaka od ovih mjera pruža uvid u različite aspekte prepoznavanja gesti i zajedno omogućuju detaljnu analizu performansi modela.

- Preciznost - označava udio ispravno prepoznatih instanci određene geste među svim instancama koje je model klasificirao kao tu gestu. Visoka preciznost znači da model rijetko pogrešno označava neku drugu gestu kao ciljnu gestu.
- Odziv - predstavlja udio ispravno prepoznatih instanci određene geste među svim stvarnim pojavama te geste u skupu podataka. Drugim riječima, odziv pokazuje koliko je model „osjetljiv” na stvarne pojave određene geste i koliko često ih uspijeva prepoznati.
- F1-mjera je harmonijska sredina preciznosti i odziva i koristi se kao jedinstvena mjera koja balansira oba aspekta; i preciznost detekcije i osjetljivost modela. F1-mjera je posebno korisna kada je važno imati ravnotežu između broja lažno pozitivnih i lažno negativnih prepoznavanja.
- Točnost - izražava udio točno klasificiranih instanci (bez obzira na klasu) u odnosu na ukupan broj primjera u evaluacijskom skupu. Točnost daje opći dojam o performansama modela, no može biti manje informativna u slučaju neuravnoteženih skupova podataka gdje su neke geste znatno brojnije od drugih.

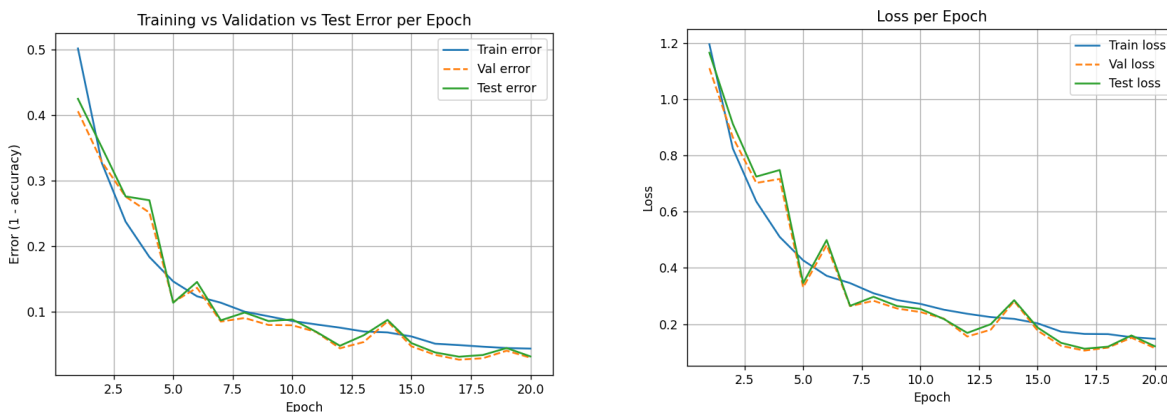
Kombiniranjem ovih mjera dobiva se cjelovita slika o kvaliteti sustava za prepoznavanje gesti, čime se omogućuje prepoznavanje njegovih prednosti i potencijalnih slabih točaka u stvarnim uvjetima rada.



## 6.2. Evaluacija istreniranog CNN-a

Evaluacija modela predstavlja ključnu fazu u procesu dubokog učenja, jer omogućava uvid u to koliko dobro konvolucijska neuronska mreža generalizira na novim, neviđenim podacima. U ovom slučaju, model je treniran na skupu gesta ruke i testiran kroz tri aspekta: praćenje pogreške i gubitka (funkcije koštanja) po epohama, izgradnju matrice zabune te izračun klasičnih klasifikacijskih metrika objašnjenih u prethodnom poglavlju.

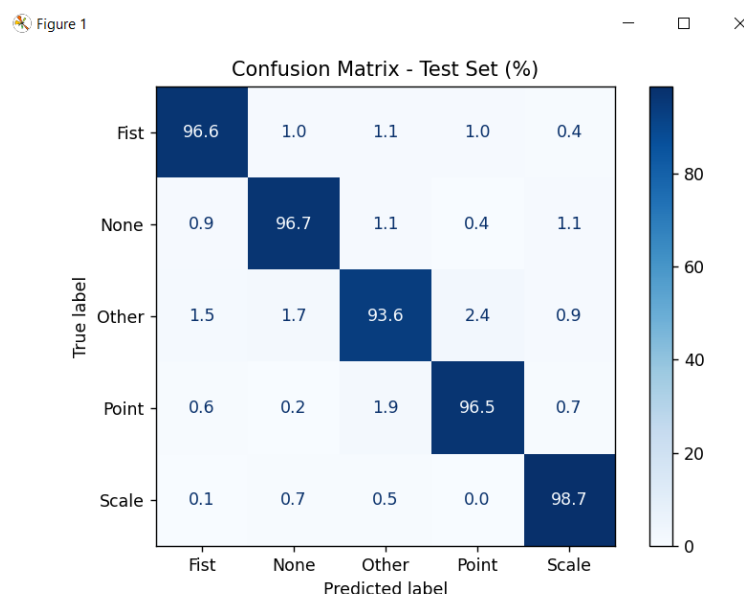
Na grafovima treniranja na slici 6.1 može se jasno vidjeti da se i pogreška i gubitak postepeno smanjuju s povećanjem broja epoha. Već u ranim epohama dolazi do naglog pada, što ukazuje da je mreža brzo naučila osnovne obrasce u podacima. Nakon otprilike 10 epoha krivulje se stabiliziraju, a razlika između trening, validacijskih i test vrijednosti ostaje minimalna. To sugerira da nije došlo do značajnijeg pretreniranja. Model ne uči napamet samo trening primjere, već zadržava sposobnost generalizacije na nepoznatim podacima. Također, činjenica da su validacijske i test krivulje vrlo blizu trening krivulji ukazuje na dobru regularizaciju i adekvatnu kompleksnost mreže.



Slika 6.1. Grafovi treniranja

Matrica zabune na slici 6.2. dodatno pojašnjava gdje model pravi najviše grešaka. Dijagonalne vrijednosti su izuzetno visoke (između 93 % i 99 %), što znači da je model gotovo uvijek točno klasificirao pripadnost svake geste. Male pogreške se pojavljuju kod klase *Other*, gdje je dio primjera ponekad pogrešno klasificiran kao *None* ili *Point*. Ovo je i očekivano jer kategorija *Other*

često obuhvaća heterogene ili manje prepoznatljive geste, pa se lako miješa s drugima. Ostale klase poput *Scale* i *Point* pokazuju gotovo savršenu prepoznatljivost s točnošću iznad 96 %, što potvrđuje da model pouzdano razlikuje standardizirane i jasne pokrete.



Slika 6.2. Matrica zabune

Rezultati klasifikacijskog izvještaja u tablici 6.1 dodatno potvrđuju zaključke iz matrice. Sve klase postižu F1-mjeru iznad 0,92, što je vrlo visok rezultat. S obzirom na to da je F1-mjera harmonijska sredina preciznosti i odziva, ovi rezultati sugeriraju da model ne pati od velikih nesrazmjera između broja točnih pozitivnih i negativnih klasifikacija. Makro prosjek metrika se kreće između 0,95 i 0,97, što je pokazatelj konzistentne performanse preko svih klasa.

Tablica 6.1. Rezultati klasifikacijskog izvještaja

| Klasa | Preciznost | Odziv  | F1-mjera |
|-------|------------|--------|----------|
| Fist  | 0,9686     | 0,9659 | 0,9673   |
| None  | 0,9498     | 0,9665 | 0,9581   |
| Other | 0,9140     | 0,9358 | 0,9248   |
| Point | 0,9842     | 0,9654 | 0,9747   |
| Scale | 0,9823     | 0,9866 | 0,9844   |

Na osnovu svih pokazatelja, CNN model demonstrira vrlo visoku točnost i stabilnost tijekom treniranja. Ne postoje jasni znakovi pretreniranja, a rezultati na testnom skupu potvrđuju sposobnost generalizacije. Blago slabija izvedba na klasi *Other* ukazuje na prostor za buduće poboljšanje, moguće kroz povećanje broja primjera te klase ili korištenje tehnika augmentacije specifičnih za nejasne geste. Ipak, u cjelini, evaluacija pokazuje da model ispunjava kriterije za pouzdano prepoznavanje gesta i da bi mogao biti primijenjen u stvarnim aplikacijama.

### **6.3. Prednosti i ograničenja sustava**

Razvijeni sustav za prepoznavanje gesta ruke temeljen na konvolucijskim neuronskim mrežama pokazuje niz značajnih prednosti. Prije svega, omogućuje intuitivnu i prirodnu interakciju s računalom bez potrebe za dodatnom opremom, što ga čini pristupačnim i jednostavnim za upotrebu u svakodnevnim uvjetima. Sustav također radi u stvarnom vremenu, što znači da korisnik dobiva trenutnu povratnu informaciju i fluidno upravljanje postavkama računala. Zahvaljujući ugrađenim mehanizmima poput histereze i pragova, osigurana je otpornost na šum i slučajne greške, čime je smanjena mogućnost neželjenih aktivacija ili pogrešnih komandi.

Unatoč visokoj učinkovitosti, sustav se suočava s određenim ograničenjima. Najizraženiji izazov odnosi se na osjetljivost na vanjske uvjete, posebno promjene u osvjetljenju i složene pozadine, koje mogu otežati pouzdanu detekciju i segmentaciju ruke. Također, varijabilnost među korisnicima, uključujući razlike u veličini, obliku i boji kože ruke, može utjecati na preciznost klasifikacije. Sustav zahtijeva optimizaciju algoritama kako bi se smanjila latencija i osigurao dosljedan rad pri složenijim gestama ili bržim pokretima. Iako su performanse u cjelini vrlo dobre, dodatni razvoj u smjeru robusnosti i prilagodljivosti različitim korisnicima i okruženjima ključan je za široku praktičnu primjenu.

## 7. Zaključak

Tijekom posljednjih godina, interakcija između čovjeka i računala doživjela je značajnu evoluciju, potaknuta razvojem novih tehnologija i pristupa. Klasične metode upravljanja, poput tipkovnice i miša, sve češće se nadopunjuju ili zamjenjuju naprednijim, intuitivnijim rješenjima. U tom je kontekstu prepoznavanje gesta ruke, kao prirodan način komunikacije, prepoznato kao iznimno potentno područje istraživanja. Ovaj diplomski rad dao je svoj doprinos razvoju upravo takvih tehnologija, demonstrirajući kako integracija računalnog vida i dubokog učenja može rezultirati učinkovitim i pouzdanim sustavima za gestualno upravljanje.

Sustav razvijen u sklopu ovog rada pokazao je da je moguće ostvariti visoku razinu preciznosti i robusnosti u prepoznavanju različitih gesta, čak i u realnom vremenu. Time je potvrđeno da suvremene metode dubokog učenja, osobito konvolucijske neuronske mreže, pružaju snažan alat za rješavanje izazova vezanih uz vizualnu klasifikaciju i interpretaciju kompleksnih obrazaca iz slike. Posebno treba istaknuti da su ostvarene performanse rezultat pažljivo dizajnirane arhitekture modela, optimiziranih parametara treniranja i primjene tehnika regularizacije, što ukazuje na važnost cjelovitog pristupa u razvoju ovakvih sustava.

Unatoč uspjehu, kroz praktičnu implementaciju i evaluaciju sustava identificirana su i određena ograničenja. Najznačajnija prepreka odnosi se na varijabilnost uvjeta osvjetljenja i kompleksnost pozadinske scene, što može otežati preciznu detekciju i segmentaciju ruke. Također, sustav pokazuje određenu osjetljivost na individualne razlike među korisnicima, poput veličine ruke, boje kože ili brzine izvođenja pokreta. Takvi izazovi naglašavaju nužnost kontinuiranog usavršavanja algoritama i proširenja baze podataka na raznovrsnije skupove primjera, kako bi se omogućila potpuna generalizacija i široka primjenjivost sustava u realnim scenarijima.

Mogućnosti za nadogradnju sustava i daljnji razvoj su brojne. Primjena multimodalnih senzora, poput kombinacije RGB i dubinskih kamera, može znatno povećati robusnost sustava na promjene okoline i omogućiti pouzdaniju detekciju gesta u složenim uvjetima. Uvođenje dinamičkih gesta, odnosno prepoznavanje sekvenci pokreta kroz vrijeme, otvorilo bi vrata još bogatijim i prirodnijim oblicima interakcije. Osim toga, *transfer learning* i korištenje unaprijed treniranih modela, poput ResNet ili EfficientNet, može dodatno povećati točnost klasifikacije i ubrzati razvoj novih rješenja.

Dugoročno gledano, ovakvi sustavi imaju potencijal transformirati način na koji ljudi komuniciraju s digitalnim uređajima. Njihova primjena nije ograničena samo na upravljanje računalima, već se može proširiti na područja pametnih kuća, asistivnih tehnologija za osobe s invaliditetom, proširene i virtualne stvarnosti, industrijske automatizacije te mnogih drugih domena gdje je potreban prirodan, brz i intuitivan način upravljanja.

Zaključno, integracija računalnog vida i dubokog učenja u sustave za prepoznavanje gesta predstavlja značajan iskorak prema razvoju prirodnih i učinkovitih sučelja za ljudsko-računalnu interakciju. Rezultati i iskustva stečena kroz ovaj rad pružaju čvrstu osnovu za daljnja istraživanja i inovacije, a sam sustav može poslužiti kao temelj za razvoj naprednih rješenja prilagođenih potrebama suvremenog društva. Otvaranjem novih mogućnosti interakcije i povećanjem dostupnosti tehnologije, ovakva rješenja mogu imati značajan društveni, tehnološki i ekonomski utjecaj, potvrđujući važnost i aktualnost istraživanja provedenog u ovom diplomskom radu.

## Literatura

- [1] Thakare, R. i dr.: "Dynamic and Static Gesture Recognition System Using Moments", International Journal of Advanced Engineering Research and Science (IJAERS), Vol. 4, No. 2, pp. 69-72, Pune, 2017.
- [2] Mitra, S.; Acharya, T.: "Gesture Recognition: A Survey", IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), Vol. 37, No. 3, pp. 311–324, 2007.
- [3] EST Engineering Systems Technologies GmbH & Co. KG: "Data Gloves", s Interneta, <https://www.est-kl.com/data-gloves/>, 13.8.2025.
- [4] Lindgren, K. i dr.: "Learned Hand Gesture Classification through Synthetically Generated Training Samples", IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 3937-3942, Madrid, 2018.
- [5] Geebelen, G. i dr.: "Real-time hand tracking with a colored glove", Proceedings of 3D Stereo Media 2010, IEEE, Liege, 13.8.2025.
- [6] Ren, Z.; Meng, J.; Yuan, J.: "Depth camera based hand gesture recognition and its applications in human-computer-interaction", Proceedings of 8th International Conference on Information, Communications & Signal Processing (ICICS), pp. 1–5, Singapore, 2011.
- [7] Maqueda Nieto, A. I.: "Development of a Hand-Gesture Recognition System for Human-Computer Interaction", Diplomski rad, School of Telecommunications Engineering, Technical University of Madrid, Madrid, 2014.
- [8] Choudhury, A.; Talukdar, A. K.; Sarma, K. K.: "A Review on Vision-Based Hand Gesture Recognition and Applications", Chapter 11 in Intelligent Applications for Heterogeneous System Modeling, IGI Global, pp. 256–281, 2015.

- [9] Solomon, C.; Breckon, T.: "Fundamentals of Digital Image Processing: A Practical Approach with Examples in MATLAB", Wiley-Blackwell, Chichester, 2010.
- [10] Salleh, T. S.; Zin, Z. M.; et al.: "Linking Mathematics and Image Processing Through Common Terminologies", Procedia – Social and Behavioral Sciences, Vol. 90, pp. 698–705, Elsevier, November 2013.
- [11] Chakraborty, B. K. i dr.: "Review of constraints on vision-based gesture recognition for human–computer interaction", IET Computer Vision, Vol. 12, No. 1, pp. 3–15, Institution of Engineering and Technology (IET), November 2017.
- [12] Gonzalez, R. C.; Woods, R. E.: "Digital Image Processing, 3rd Edition", Pearson Prentice Hall, Upper Saddle River, 2008.
- [13] Szeliski, R.: "Computer Vision: Algorithms and Applications, 2nd Edition", Springer, Cham, 2022.
- [14] Augmented Startups: "Computer Vision vs. Human Vision: Unveiling the Battle of Perception", s Interneta, <https://shorturl.at/R7l02>, 22. kolovoza 2025.
- [15] Kakumanu, P.; Makrogiannis, S.; Bourbakis, N.: "A survey of skin-color modeling and detection methods", Pattern Recognition, vol. 40, no. 3, pp. 1106–1122, Elsevier, Amsterdam, 2007.
- [16] Sarma, D.; Bhuyan, M. K.: "Hand Gesture Recognition Using Deep Network Through Trajectory-to-Contour Based Images", 2018 15th IEEE India Council International Conference (INDICON), IEEE, pp. 1-6, Coimbatore (Indija), 2018.
- [17] Mod Ma'asum, F. F.; Sulaiman, S.; Saparon, A.: "Comparative Study of Segmentation Technique for Hand Gesture Recognition System", Journal of Telecommunication, Electronic and Computer Engineering (JTEC), Vol. 9, No. 2–2, pp. 7–10, 2017.
- [18] Ramamoorthy, A. i dr.: "Recognition of Dynamic Hand Gestures", Pattern Recognition, Vol. 36, No. 9, pp. 2069–2081, Elsevier Science Ltd., 2003.

- [19] Yeo, A.: "Kalman Filter", s Interneta, [https://angeloyeo.github.io/2021/04/07/Kalman\\_filter\\_en.html](https://angeloyeo.github.io/2021/04/07/Kalman_filter_en.html), 23.8.2025.
- [20] Dimitrov, M.: "Particle Filters", Lancaster University, <https://www.lancaster.ac.uk/stor-i-student-sites/martin-dimitrov/2021/05/14/particle-filters/>, 23.8.2025.
- [21] Bisht, D. i dr.: "Smart Communication System Using Sign Language Interpretation", Proceedings of the 31st Conference of FRUCT Association, IEEE, Helsinki, 2022..
- [22] Ng, A.; Ma, T.: "CS229 Lecture Notes", Stanford University, [https://cs229.stanford.edu/main\\_notes.pdf](https://cs229.stanford.edu/main_notes.pdf) , 24.08.2025.
- [23] LeCun, Y.; Bengio, Y.; Hinton, G.: "Deep learning", Nature, Vol. 521(7553), pp. 436–444, London, 2015.
- [24] Lawtomed: "A.I. Technical: Machine vs Deep Learning", Lawtomed, s Interneta, <https://lawtomed.com/a-i-technical-machine-vs-deep-learning/>, 24.08.2025
- [25] Stanford: "Convolutional Neural Networks (CNNs) - Course Notes", CS231n: Deep Learning for Computer Vision, Stanford University, s Interneta, <https://cs231n.github.io/convolutional-networks/>, 25.08.2025.
- [26] DataHacker.rs: "#003 CNN - More on Edge Detection", Master Data Science, s Interneta, <https://datahacker.rs/edge-detection-extended/>, 25.08.2025.
- [27] Dumoulin, V.; Visin, F.: "A guide to convolution arithmetic for deep learning", s Interneta, <https://arxiv.org/pdf/1603.07285>, 25.08.2025.



## Sažetak

U ovom radu istražuje se razvoj sustava za prepoznavanje gesta ruke temeljenog na računalnom vidu i konvolucijskim neuronskim mrežama, s ciljem omogućavanja prirodnije i intuitivnije interakcije čovjeka s računalom. Sustav koristi kameru za akviziciju slike, algoritme za detekciju i segmentaciju ruke te duboke neuronske mreže za klasifikaciju gesta u stvarnom vremenu. Teorijski okvir rada obuhvaća ključne pojmove iz područja računalnog vida i strojnog učenja, s posebnim naglaskom na arhitekturu i funkcionalnost konvolucijskih neuronskih mreža, koje su se pokazale najprikladnijim pristupom za zadatke vizualne klasifikacije. U praktičnom dijelu razvijen je prototip sustava koji postiže visoku razinu preciznosti, pri čemu većina klasa gesta ostvaruje F1-mjeru iznad 0,95, uz zadržavanje brzine obrade potrebne za rad u stvarnom vremenu. Analiza metrika, grafova gubitka i točnosti te matrice zabune pokazuje da sustav nije sklon pretreniranju i može generalizirati na neviđene primjere. Identificirana ograničenja uključuju osjetljivost na promjene osvjetljenja, kompleksnost pozadine i individualne razlike među korisnicima, što upućuje na potrebu za daljnjim proširenjem baze podataka i optimizacijom algoritama. Mogući smjerovi budućeg razvoja uključuju uvođenje multimodalnih senzora, primjenu naprednijih arhitektura dubokih mreža te proširenje skupa na dinamičke geste, čime bi se dodatno povećala robusnost i funkcionalnost sustava. Rezultati potvrđuju da računalni vid i duboko učenje mogu uspješno zamijeniti tradicionalne metode upravljanja i otvoriti put novim oblicima ljudsko-računalne interakcije. Razvijeni sustav predstavlja čvrstu osnovu za daljnja istraživanja i širu primjenu u područjima poput pametnih kuća, asistivnih tehnologija i virtualne stvarnosti, naglašavajući važnost i aktualnost ovakvih rješenja u suvremenom digitalnom okruženju.

**Ključne riječi:** Računalni vid, konvolucijske neuronske mreže, prepoznavanje gesta, gestualno upravljanje

## Summary

### **TITLE: Application of Computer Vision and Convolutional Neural Networks for Human Gesture Recognition with the Aim of Gestural Control of Computer Settings**

This thesis explores the development of a hand gesture recognition system based on computer vision and convolutional neural networks, with the goal of enabling a more natural and intuitive human–computer interaction. The system employs a camera for image acquisition, algorithms for hand detection and segmentation, and deep neural networks for real-time gesture classification. The theoretical framework covers key concepts from the fields of computer vision and machine learning, with a particular emphasis on the architecture and functionality of convolutional neural networks, which have proven to be the most suitable approach for visual classification tasks. In the practical part, a prototype system was developed that achieves a high level of accuracy, with most gesture classes reaching an F1-score above 0.95, while maintaining the processing speed required for real-time operation. The analysis of metrics, loss and accuracy graphs, and the confusion matrix shows that the system is not prone to overfitting and can generalize to unseen examples. Identified limitations include sensitivity to lighting changes, background complexity, and individual differences among users, pointing to the need for further dataset expansion and algorithm optimization. Possible directions for future development include the introduction of multimodal sensors, the application of more advanced deep network architectures, and the extension of the dataset to dynamic gestures, which would further enhance the robustness and functionality of the system. The results confirm that computer vision and deep learning can successfully replace traditional control methods and pave the way for new forms of human–computer interaction. The developed system provides a solid foundation for further research and broader application in areas such as smart homes, assistive technologies, and virtual reality, highlighting the importance and relevance of such solutions in the modern digital environment.

**Key words:** Computer Vision, Convolutional Neural Networks, Gesture Recognition, Gestural Control